Presentation De Kubernetes

- Nous avons vu jusqu'à présent dans le chapitre `Docker` que nous pouvons créer et exécuter des conteneurs sur l'hôte `Docker`.
- C'est tellement cool, c'est léger, rapide et livrable, mais qu'en est-il de la production.
- Pouvons-nous exécuter nos applications sur ces conteneurs légers ou, plus spécifiquement, vous demander d'exécuter tous ces conteneurs sur un seul hôte ?
- La réponse est qu'il n'est pas prudent de mettre tous ses œufs dans le même panier.
- Nous avons donc besoin d'un `cluster d'hôtes Docker`, qui peut être géré par une application externe.
- Quelque chose qui puisse planifier des `conteneurs` pour nous sur l'hôte le mieux approprié du `cluster`.
- Il devrait également détecter un `conteneur` défaillant et résoudre le problème pour nous.
- Nous recherchons donc un `outil d'orchestration Docker`.

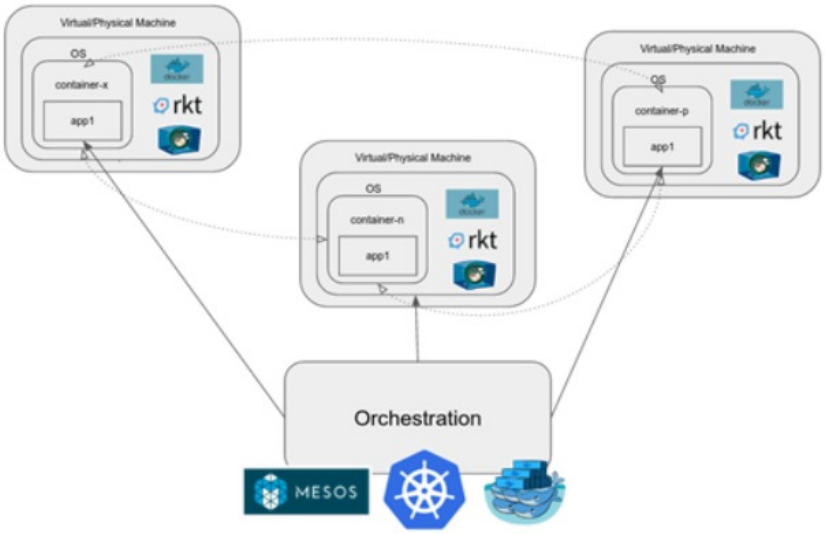


Figure 1: Alt Text

- Nous disposons de peu `d'outils d'orchestration Docker et de services cloud` sur le marché, comme indiqué ci-dessous.
  - `Amazon ECS : Amazon EC2 Container Service (ECS)` prend en charge les `conteneurs Docker` et vous permet d'exécuter des applications sur un `cluster` géré d'instances `Amazon EC2`.

- Azure Container Service (ACS) : ACS vous permet de créer un cluster de machine virtuelles qui agissent en tant qu'hôtes de conteneurs ainsi que de machines maîtres utilisées pour gérer vos conteneurs d'applications.
- Cloud Foundry's Diego : Diego est un système de gestion de conteneurs qui combine un planificateur, un exécuteur et un gestionnaire de santé. Il s'agit d'une réécriture Cloud Foundry.
- CoreOS Fleet - Fleet est un outil de gestion de conteneurs qui vous permet de déployer des conteneurs Docker sur des hôtes d'un cluster ainsi que de distribuer sur un cluster
- Docker Swarm : Docker Swarm fournit unr fonctionnalité de clustering native pour les conteneurs Docker, qui vous permet de transformer un groupe de moteurs Docker en un seul moteur Docker virtuel.
- Google Container Engine - Google Container Engine, basé sur Kubernetes, vous permet d'exécuter des conteneurs Docker sur la plate-forme Google Cloud. Il planifie les conteneurs dans le cluster et les gère en fonction des exigences définies par l'utilisateur.
- Kubernetes - Kubernetes est un système d'orchestration pour les conteneurs Docker Il gère la planification et gère les charges de travail en fonction de paramètres définis par l'utilisateur.
- Mesosphère Marathon - Marathon est un framework d'orchestration de conteneurs pour Apache Mesos conçu pour lancer des applications de longue durée. Il offre des fonctionnalités clés pour exécuter dans un environnement en cluster.

Pourquoi Kubernetes ?

- Parmi eux, nous nous sur Kubernetes dans ce didacticiel pour les raisons mentionnées ci-dessous.
  - Kubernetes est le propre projet de Google qu'ils ont utilisé pour gérer les conteneurs des 10 dernières années. Ainsi, une vaste expérience en matière d'applications et un modèle mature sont des élements que nous recherchons pour les productions.
  - Kubernetes est classé parmi les meilleurs outils d'orchestration de conteneurs du marché selon une enquête.
  - Il prend en charge d'autre plate-formes de conteneurs en dehors de Docker comme RKT.
  - Kubernetes est un projet open source, distinct des autres projets **pilotés par les fournisseurs** comme Swarm, Msos, CloudFoundry. Docker swarm est également open source mais il est étroitement intégré à d'autres outils Docker.
  - Il peut prendre en charge un grand nombre d'applications. C'est un sujet de discussion au sein de la communauté Kubernetes depuis le printemps, lorsqu'elle a annoncé que l'outil pouvait exécuter plus de

`1000 noeuds`.
  - Cela ne veut pas dire que les autres outils d'orchestration ne sont pas que `kubernetes`, mais nous voulions simplement essayer d'abord les meilleurs d'entre eux, puis nous pourrons en essayer d'autres d'il y a des lacunes avec `Kubernetes`.

Qu'est-ce que Kubernetes ?

- `Ansible` peut contrôler un grand nombre de serveurs et facilite les tâches d'administrations et d'exploration.
- `Ansible` peut effectuer une gestion de configuration simple et une orchestration complexe, il possède toutes les fonctionnalités des outils de configuration et il est très facile à apprendre et à mettre en oeuvre.
- Il communique via les `canaux SSH` normaux afin de récupérer des informations sur des machines distantes, d'émettre des commandes et de copier des fichiers. Pour les noeud `Windows`, il utilise `winrm`.

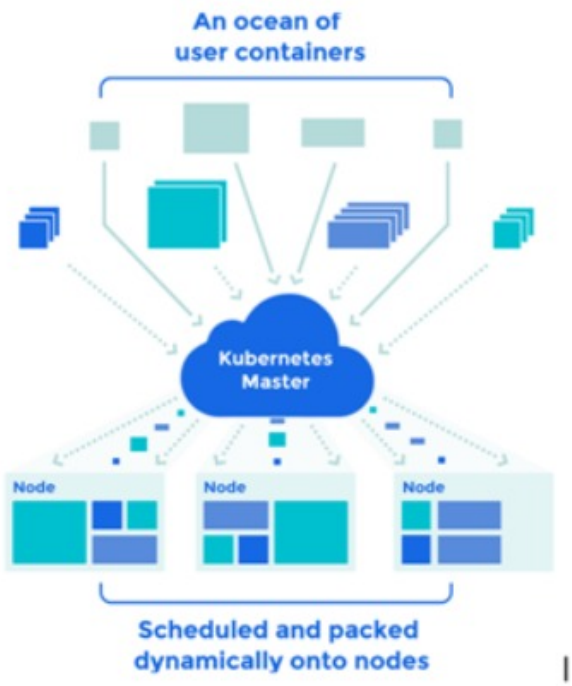


Figure 2: Alt Text

Selon La documentation Kubernetes

- `Kubernetes` est une `plate-forme open source` permettant `d'automatiser ,la mise en échelle et les opérations de conteneurs d'applications sur des clusters d'hôtes`, fournissant une infrastructure centrée sur les conteneurs.
- Avec `Kubernetes`, vous pouvez répondre rapidement et efficacement à la demande des clients:
  - `Déployez` vos applications de manière rapide et prévisible.
  - Faites évoluer vos applications à la volée.
  - Déployez de nouvelles fonctionnalités en toute transparence.
  - Limitez l'utilisation du matériel aux ressources requises uniquement.
- L'objectif de `Kubernetes` est de favoriser un écosystème de composants et d'outils qui allègent le fardeau d'applications dans les `cloud publics et privés`.
- Kubernetes est:
  - `Portable` : public, privé, hybride, multi-cloud.
  - `Extensible` : modulaire, enfichable, hookable, composable.
  - `Auto-réparation` : placement automatique, redémarrage automatique, replication automatique, mise à l'échelle automatique.
  - `Google` a lancé le projet `Kubernetes` en 2014. `Kubernetes` s'appuie sur une décennie et demie d'expérience de `Google` dans l'exécution de charges de travail de production à grande échelle, combiné aux meilleures idées et pratiques de la communauté.

Que peut faire Kubernetes ?

- Au minimum, `Kubernetes` peut planifier et exécuter des conteneurs d'applications sur des `clusters de machines physiques ou virtuelles`.
- `Kubernetes` fournit l'infrastructure nécessaire pour créer un environnement de développement véritablement centré sur les conteneurs.
- `Kubernetes` répond à un certain nombre de besoins courants des applications exécutées en production, telles que :
  - `Colocalisation des processus d'assistance`, facilitant les applications composites et préservant le modèle d'une application par conteneur.
  - Montage de systèmes de stockage.
  - Distribution de secrets.
  - Vérification de l'état des applications
  - Replication instances d'application
  - Utilisation de la mise à l'échelle automatique des pods horizontaux.
  - Dénomination et découverte

  - Equilibrage des charges
  - Mise à jour progressive.
  - Surveillance des ressources
  - Accès et ingestion de journaux
  - Débogage d'applications
  - Fournir une authentification et une autorisation

Architecture Kubernetes

- `Kubernetes` est une pile de services qui fonctionnent ensemble pour gérer tous les hôtes, le cluster `Kubernetes` est ainsi que nous appelons ensemble `AKA K8`.
- Tous ces services ou composants sont présentés ci-dessous.
  - `POD` : un `pod` est un groupe d'un ou plusieurs conteneurs (tels que des conteneurs `Docker`), le stockage partagé pour ces conteneurs et des options sur la façon d'exécuter les conteneurs. Les `pods` sont toujours colocalisés et co-programmés, et exécutés dans un contexte partagé. Un `pod` modélise un `hôte logique` spécifique à une application. Il contient un ou plusieurs conteneurs d'applications relativement étroitement couplés - dans un monde pré-conteneur, lis se seraient exécutés sur la même machine physique ou virtuelle.
  - `SERVICE` : Les pods `kuberenetes` sont mortels. Il naissent et lorsqu'ils meurent, ils ne ressuscitent pas. Les `ReplicationControllers` en particulier créent et détruisent de manière dynamique (par exemple lors d'une mise à l'échelle vers le haut ou vers le bas ou lors de mises à jour progressive). Bien que chaque `pods` ait sa propre `addresse IP`, même ces addresses IP ne peuvent pas être fiables pour être stables dans le temps. Cela conduit à un problème. Le `service` sera au-dessus du `POD` et aura une adresse IP stable, c'est comme un `équilibreur de charge` donc peu importe les `noeuds` que vous avez sous l'équilibreur de charge et quelle est leur adresse IP, l'équilibreur de charge pourra y accéder. C'est similaire pour le `service` dans `Kubernetes`.

Architecture du Noeud Maître

- `etcd` : Il s'agit d'un magasin clé-valeur open source developpé par `CoreOS`. `Kubernetes` utilise `etcd` pour stocker les données de configuration accessibles par tous les noeuds (sbires et maître) du cluster. Des exemples de données stockées par `Kubernetes` dans `etcd` sont les tâches planifiées, es détails et l'état du `pod/service` crées et déployés, les espaces de noms et les informations de réplication etc.
- `KubeApiServer` : Le serveur `API Kubernetes` valide également générale-

ment le magasin de données de configuration dans `etcd` et les détails du conteneur déployé qui sont en accord. Il fournit également une interface `RESTFUL` pour faciliter la communication.

- `Serveur Kube-Schedule` : le déploiement `des pods et des services` configurés sur les noeuds est effectués par le composant planificateur. Il est responsable de l'attribution des tâches aux niveaux du cluster. Le planificateur dispose également des informations concernant les ressources sur les membres du cluster. Comme ceux requis pour l'exécution du service configuré et est donc en mesure de décider où déployer un service spécifique.
- `Kube-Controller-Manager` : Il est généralement responsable de la gestion des fonctions au niveau du `cluster` telles que le `contrôleur de réplication`. Chaque fois que l'état souhaité du cluster change, il est écrit dans `etcd` puis le gestionnaire de contrôleur tente de faire apparaître le `cluster` dans l'état souhaité. Un contrôleur utilise `apiserver` pour surveiller l'état partagé du cluster et apporte des modifications correctives à l'état actuel pour qu'il corresponde à celui souhaité. Un exemple est `Replication Controller` qui prend en charge les `PODS` dans le système, si un `POD` échoue, il le remplace par un nouveau `POD`.

Architectures des Noeuds de serviteurs.

- `Docker` : L'une des exigences de base des neouds est `Docker`.`Docker` est responsable de l'extraction et de l'exécution du conteneur à partir des images `Docker`.
- `Kube-proxy` : chaque noeud du cluster exécute un simple proxy réseau. L'utilisation d'un noeud proxy dans le cluster achemine la requête vers le conteneur approprié dans le noeud.
- `kubelet` : Il s'agit d'un processus agent qui s'exécute suer chaque noeud. Il est responsable de la gestion `des pods et leurs conteneurs`. Il traite les spécifications des pods qui snt définies au format `YAML ou JSON`. `Kubelet` prend les spécifications des pods et vérifie si les pods fonctionnent correctement ou non.
- `Flannel` : Il s'agit d'un réseau superposé qui fonctionne sur l'attribution d'une plage d'adresses de sous-réseau. Il est utilisé pour attribuer des `adresses IP` à chaque `pod` exécuté et pour établir les communications de pod à pod et de pod à services.

Configuration de Kubernetes

- Kubernetes peut être configuré sur une seule machine `Linux` ou nous pouvons avoir `le maître et les noeuds` sur une machine séparée.
- Comme vous l'auriez deviné, un cluster à noeud unique est idéal pour l'apprentissage et les tests, mais un cluster `Kubernetes` de qualité de production est nécessaire pour gérer des applications et des conteneurs

à grande échelle. Nous verrons les deux manières de mettre en place un `cluster Kubernetes`. La configuration manuelle de tous les composants `Kubernetes` est une tâche très fastidieuse, mais il existe peu d'outils sur le marché capable d'automatiser le déploiement du cluster `Kubernetes` pour nous.

  - `Minikube` : `Minikube` configure un cluster à noeud unique sur une machine virtuelle exécuté sur `viurtualbox`. Nous pouvons créer et également géré le cluster `Kubernetes avec minikube`.
  - `Kops` : `Kops` met en place un cluster kubernetes multi-neuds de qualité de production sur le cloud `AWS`, actuellement il ne prend en charge que le fournisseur `AWS`.

Configuration du Minikube Localement

Qu'est-ce qu'un Minikube ?

- `Minikube` exécute un cluster `kubernetes` à noeud unique dans une `VM` sur votre ordinateur portable.
- C'est pour apprendre à tester `Kubernetes`, il ne doit pes être utilisé en production.

Configurez Minikube

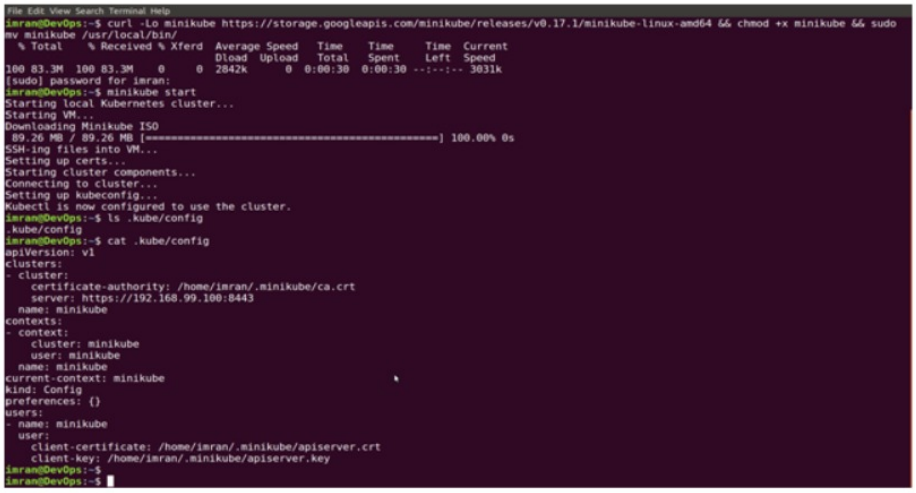
- `Minikube` peut être télécharger depuis son dépôt github : https://github.com/kubernetes/releases.
- Nous allons configurer `minikube` sur une machine `Linux`, alors vérifiez `l'URL` de la dernière version et téléchargez le package.
- `L'URL` mentionnée dans la capture d'écran était la plus récente au moment de la préparation du document `Virtualbox` en est la dépendance, car par défaut, il créera une machine virtuelle sur `virtualbox` et y configurera un `minikube`.

Kubectl

- `kubectl` est une interface de ligne de commande permettant d'exécuter des commandes sur des clusters `Kubernetes`.

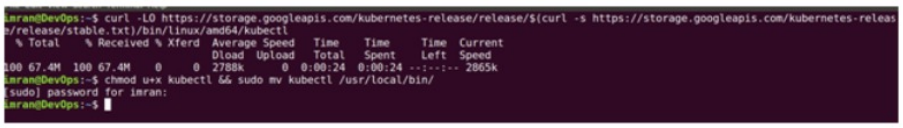
Installation et configuration de Kubectl

- Consultez la page de téléchargement pour connaitre l'URL de la dernière version : https://kubernetes.io/docs/tasks/kubectl/install/
- `Kubectl` lira la configuration à partir de `~/.kube/config` et connaîtra `l'IP`, le port, l'authentification et d'autres détails pour se connecter au cluster `Kubernetes`.
- Exécutez la commande `kubectl` pour vérifier si elle fonctionne.



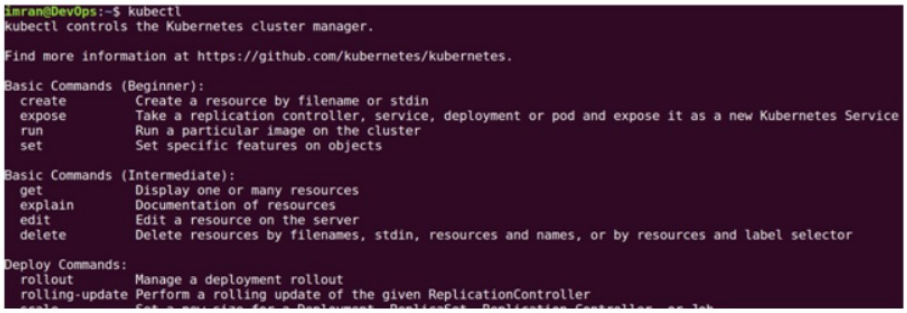
```
File Edit View Search Terminal Help
imran@DevOps:~$ curl -Lo minikube https://storage.googleapis.com/minikube/releases/v0.17.1/minikube-linux-amd64 && chmod +x minikube && sudo
mv minikube /usr/local/bin/
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 83.3M  100 83.3M    0     0  2842k      0  0:00:30  0:00:30 --:--:-- 3031k
[sudo] password for imran:
imran@DevOps:~$ minikube start
Starting local Kubernetes cluster...
Starting VM...
Downloading Minikube ISO
 89.26 MB / 89.26 MB [============================================] 100.00% 0s
SSH-ing files into VM...
Setting up certs...
Starting cluster components...
Connecting to cluster...
Setting up kubeconfig...
Kubectl is now configured to use the cluster.
imran@DevOps:~$ ls .kube/config
.kube/config
imran@DevOps:~$ cat .kube/config
apiVersion: v1
clusters:
- cluster:
    certificate-authority: /home/imran/.minikube/ca.crt
    server: https://192.168.99.100:8443
  name: minikube
contexts:
- context:
    cluster: minikube
    user: minikube
  name: minikube
current-context: minikube
kind: Config
preferences: {}
users:
- name: minikube
  user:
    client-certificate: /home/imran/.minikube/apiserver.crt
    client-key: /home/imran/.minikube/apiserver.key
imran@DevOps:~$
imran@DevOps:~$
```

Figure 3: Alt Text



```
imran@DevOps:~$ curl -LO https://storage.googleapis.com/kubernetes-release/release/$(curl -s https://storage.googleapis.com/kubernetes-releas
e/release/stable.txt)/bin/linux/amd64/kubectl
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 67.4M  100 67.4M    0     0  2788k      0  0:00:24  0:00:24 --:--:-- 2865k
imran@DevOps:~$ chmod u+x kubectl && sudo mv kubectl /usr/local/bin/
[sudo] password for imran:
imran@DevOps:~$
```

Figure 4: Alt Text



```
imran@DevOps:~$ kubectl
kubectl controls the Kubernetes cluster manager.

Find more information at https://github.com/kubernetes/kubernetes.

Basic Commands (Beginner):
  create         Create a resource by filename or stdin
  expose         Take a replication controller, service, deployment or pod and expose it as a new Kubernetes Service
  run            Run a particular image on the cluster
  set            Set specific features on objects

Basic Commands (Intermediate):
  get            Display one or many resources
  explain        Documentation of resources
  edit           Edit a resource on the server
  delete         Delete resources by filenames, stdin, resources and names, or by resources and label selector

Deploy Commands:
  rollout        Manage a deployment rollout
  rolling-update Perform a rolling update of the given ReplicationController
```

Figure 5: Alt Text

```
imran@DevOps:~$ kubectl run hello-minikube --image=gcr.io/google_containers/echoserver:1.4 --port=8080
deployment "hello-minikube" created
imran@DevOps:~$ kubectl expose deployment hello-minikube --type=NodePort
service "hello-minikube" exposed
imran@DevOps:~$ minikube service hello-minikube --url
http://192.168.99.100:31130
imran@DevOps:~$
```

Figure 6: Alt Text

- Essayez les commandes dans la capture d'écran ci-dessous pour tester si le cluster fonctionne.

Figure 7: Alt Text

- Cela montre que notre cluster `Kubernetes` fonctionne, nous sommes capables d'extraire des images, d'exécuter un conteneur et également d'accéder à son service.
- Si vous souhaitez arrêter votre cluster, vous pouvez exécuter `Minikube stop` et cela fera tomber le cluster `minikube`.

KOPS

Cluster Kubernetes Sur Le Cloud AWS

Qu'est-ce que le Kops ?

- `Kops` vous permet de déployer des clusters `Kubernetes` de qualité production et hautement disponibles à partir de la ligne de commande
- Le déploiement est actuellement pris en charge sur `Amazon Web Services (AWS)`, et d'autres plates-formes sont prévues.
  - Installation, mises à niveau et gestion des opérations `Kubernetes (kops)` de niveau production `K8`.
  - Le moyen le plus simple de mettre en place et de faire fonctionner un cluster `Kubernetes` de niveau production.
  - Fonctionne sur `Mac OS/Linux`
- Nous allons configurer une machine virtuelle `Linux` avec `vagrant+virtualbox` et configurer `Kops` dans la machine virtuelle.

Kops sur la VM

- Configurer `vagrant` et `virtualbox`

- Créez un repertoire `Kops` et affichez `xenia64 vm`.
- Affichez la machine virtuelle.

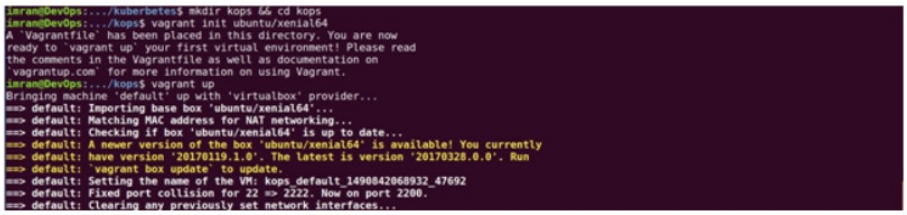


Figure 8: Alt Text

- Copiez le lien de la dernière version `Linux-amd64` de `Kops` depuis `github`.

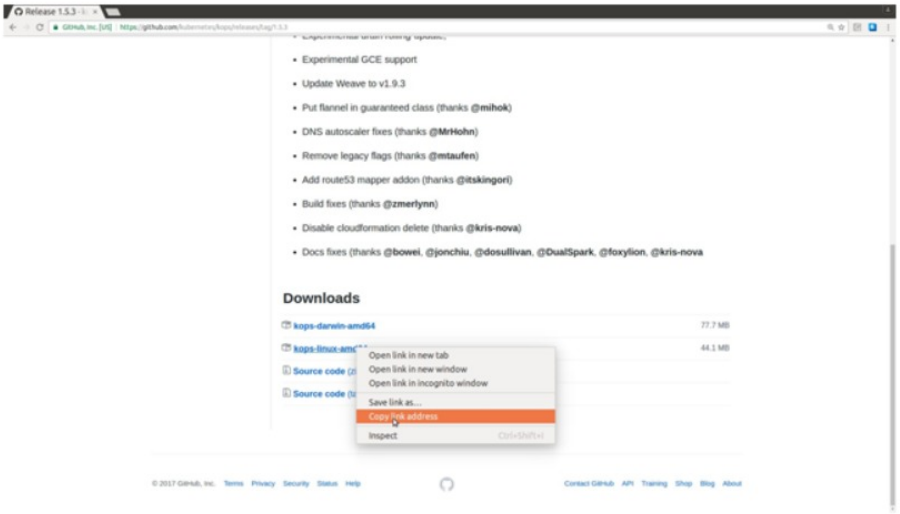


Figure 9: Alt Text

- Connectez-vous à la machine virtuelle et télécharger `Kops`.
- `make` est exécutable et déplacé vers `/usr/local/bin`.
- Installer l'interface `CLI AWS`.

Configuration AWS Pour Kops

- Créez un compartiment `S3` dans la région que vous utiliserez pour stocker l'état de `KOPS`.

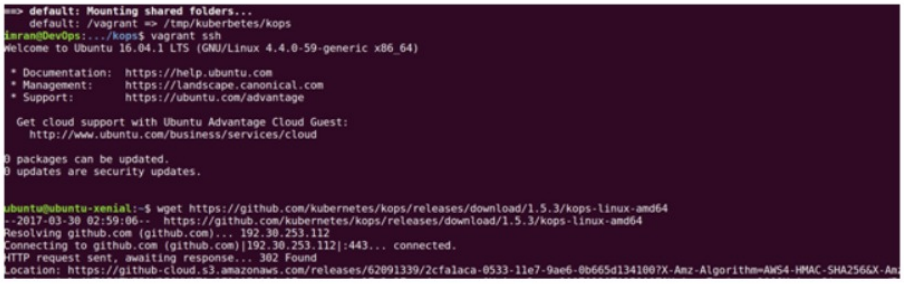


Figure 10: Alt Text

Figure 11: Alt Text

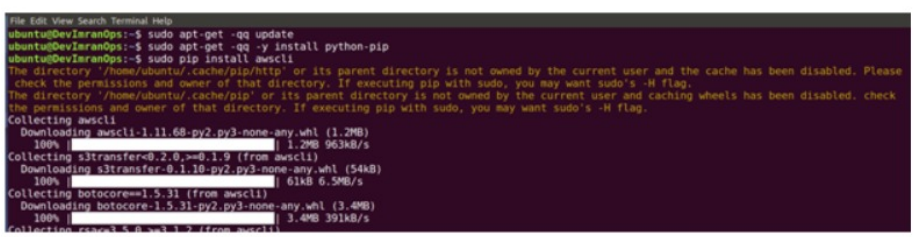


Figure 12: Alt Text

Configuration Du Nom De Domaine

- Nous avons besoin du nom de domaine et de quelques sous-domaines, nous pouvons configurer les deux avec `AWS route53`.
- Vous pouvez utiliser `route53` pour le nom de domaine et le sous-domaine si vous le souhaitez.
- Enregistrez un domaine auprès de n'importe quel fournisseur de nom de domaine `GoDaddy ou Namecheap ou Route53`.

Configuration du sous-domaine avec `AWS Route53`

- Accédez au tableau de bord `AWS route53` => `Gestion DNS` => Créer une zone hébergée.
- Nom de domaine => `kubernetes .<Votre nom de domaine>`.
- Notez les quatre valeurs `ns` que vous voyez après la création du sous-domaine.

Configuration de `NS Record` avec le fournisseur de domaine

- Accéder au site de fournisseur de domaine.
- Ajoutez un enregistrement `NS` puis le sous-domaine `Kubernetes`.

- Pour `namecheap` suivez la procédure mentionnée ci-dessous.
- `Manage` => `Advanced DNS` => `Add new record`.
- Vous devez ajouter les quatre valeurs d'enregistrement `ns` car nous avions quatre noms de serveur `ns` dans `Route53`.

Figure 13: Alt Text

- N'utilisez pas le nom du serveur `NS` mentionné dans la capture d'écran, utilisez celui que vous avez créé avec `Route53`.

Configuration d'AWS CLI

- Créez un utilisateur `IAM` avec un accès complet et téléchargez son fichier d'informations d'identification.
- Exécutez la commande `AWS` configure notre `VM` et saisissez la clé d'accès `AWS` et la clé secrète du fichier `credentials.csv`.

Configurez Kubectl

- Installation et configuration de `kubectl`.

```
ubuntu@DevImranOps:~$ sudo -i
root@DevImranOps:~# aws configure
AWS Access Key ID [None]: AKIAJXEPCUDXBJKS3B0Q
AWS Secret Access Key [None]: 19o2wCBkVDVP5fSbe7k9blSaYRKG3AvNLSarNSlL
Default region name [None]:
Default output format [None]:
root@DevImranOps:~#
```

Figure 14: Alt Text

- Vérifiez la page de téléchargement pour connaître `l'URL` de la dernière version : https://kubernetes.io/docs/tasks/kubectl/install/

```
root@DevImranOps:~# chmod u+x kubectl && mv kubectl /usr/local/bin/
root@DevImranOps:~#
```

```
root@DevImranOps:~# curl -LO https://storage.googleapis.com/kubernetes-release/release/$(curl -s https://storage.googleapis.com/kubernetes-re
lease/release/stable.txt)/bin/linux/amd64/kubectl
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 67.4M  100 67.4M    0     0  2885k      0  0:00:23  0:00:23 --:--:-- 2901k
```

Figure 15: Alt Text

- Configurer les clés `SSH` pour la connexion au cluster

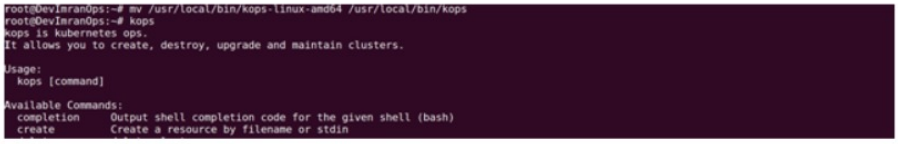
```
root@DevImranOps:~# ssh-keygen -f .ssh/id_rsa
Generating public/private rsa key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in .ssh/id_rsa.
Your public key has been saved in .ssh/id_rsa.pub.
The key fingerprint is:
SHA256:oxGZrDQSA/J/WqzSNM0rh6M5XZY+MmPUSW9YcWDbvdQ root@DevImranOps
The key's randomart image is:
+---[RSA 2048]----+
|o.o      o.      |
|.. o . +.o.. .   |
|  o o = .o. o E  |
|   + *...  . .   |
|    =oB*S   .    |
|   o.BB+o.       |
|  .oB++.         |
|  .e*+o          |
|  oo + .         |
+----[SHA256]-----+
root@DevImranOps:~#
```

Figure 16: Alt Text

- Renommer le binaire `kops` et le tester
- Configurez le serveur de noms `Google DNS (8.8.8.8, 8.8.8.4` dans notre `VM` puisse résoudre `Kubernetes.<Votre nom de domaine>`.
- `Kubernetes.<Votre nom de domaine>` doit être résolu en une valeur d'enregistrement `ns` après cela.
- Comme indiqué dans la capture d'écran ci-dessous, cela résout pour moi.

Configurer le Cluster Kubernetes

- C'est maintenant l'heure du spectacle, nous allons créer un cluster `Kubernetes` sur `AWS` avec la commande `Kops`.
- A partir de maintenant, nous devrions avoir la configuration mentionnée ci-dessous prête à partir d'`AWS`.



```
root@DevImranOps:~# mv /usr/local/bin/kops-linux-amd64 /usr/local/bin/kops
root@DevImranOps:~# kops
kops is kubernetes ops.
It allows you to create, destroy, upgrade and maintain clusters.

Usage:
  kops [command]

Available Commands:
  completion     Output shell completion code for the given shell (bash)
  create         Create a resource by filename or stdin
```

Figure 17: Alt Text

```
root@DevImranOps:~# vi /etc/network/interfaces
#VAGRANT-BEGIN
# The contents below are automatically generated by Vagrant. Do not modify.
auto enp0s8
iface enp0s8 inet dhcp
    post-up route del default dev $IFACE || true
    dns-nameservers 8.8.8.8 8.8.4.4
#VAGRANT-END
```

Figure 18: Alt Text

```
root@DevImranOps:~# nslookup -type=ns kubernetes.devimranops.club
Server:         8.8.8.8
Address:        8.8.8.8#53

Non-authoritative answer:
kubernetes.devimranops.club     nameserver = ns-1120.awsdns-12.org.
kubernetes.devimranops.club     nameserver = ns-1896.awsdns-45.co.uk.
kubernetes.devimranops.club     nameserver = ns-214.awsdns-26.com.
kubernetes.devimranops.club     nameserver = ns-639.awsdns-15.net.

Authoritative answers can be found from:

root@DevImranOps:~#
```

Figure 19: Alt Text

  - `Bucket S3`, mon nom de compartiment est `kops-state-86`.
  - Nom de domaine complet pour notre cluster, le mien est `kubenetes.devimranops.club`.
  - Configuration de l'`AWS CLI` avec un utilisateur administrateur `IAM`.
  - Région `AWS`, j'utilise `us-west-1`
  - Zone `AWS`, j'utilise `us-west-1a`

- Nous devons fournir toutes les valeurs à la commande `kops` comme indiqué ci-dessous.

```
root@DevImranOps:~# kops create cluster --name=kubernetes.devimranops.club --state=s3://kops-state-86 --zone
s=us-west-1a --node-count=2 --node-size=t2.micro --master-size=t2.micro --dns-zone=kubernetes.devimranops.cl
ub
```

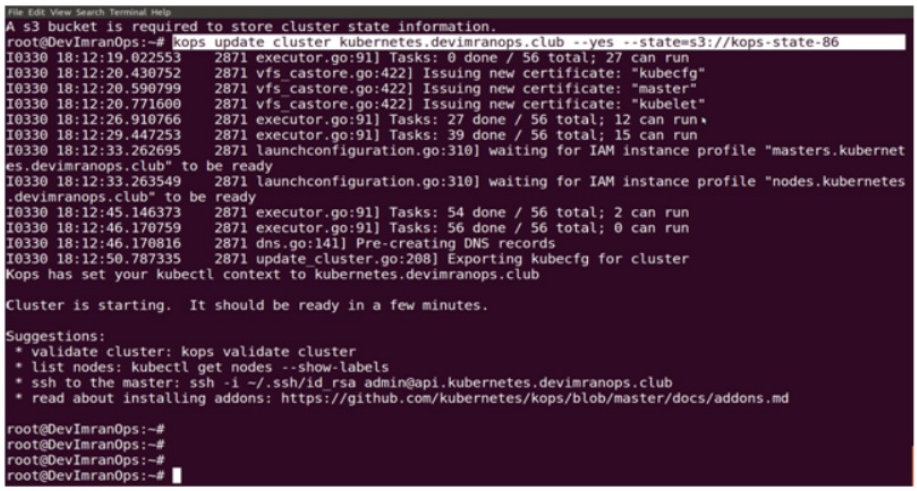
Figure 20: Alt Text

- Vous devriez obtenir le résultat ci-dessous, qui montre que la configuration du cluster est crée par `kops`.
- Le cluster n'a pas encore été créé, suivez la commande suivante pour la créer.

```
Must specify --yes to apply changes

Cluster configuration has been created.

Suggestions:
 * list clusters with: kops get cluster
 * edit this cluster with: kops edit cluster kubernetes.devimranops.club
 * edit your node instance group: kops edit ig --name=kubernetes.devimranops.club nodes
 * edit your master instance group: kops edit ig --name=kubernetes.devimranops.club master-us-west-1a

Finally configure your cluster with: kops update cluster kubernetes.devimranops.club --yes

root@DevImranOps:~#
```

Figure 21: Alt Text

- `Kops` met à jour le cluster pour créer le cluster.
- La configuration du cluster aurait été lancée, vérifiez-la via `AWS`.
- Vérifier les sous-domaine `Route53`
- Validez le cluster avec la commande `kops`
- `Kops` crée le fichier `~/.kube/config` dans lequel il écrit tous les détails de configuration pour `Kubectl`.
- Nous pouvons maintenant commencer à utiliser la commande `kubectl` pour interagir avec notre cluster `kubernetes`.
- Vérifiez tous les noeuds du cluster.
- Exécutons un conteneur sur ce cluster
- Déploiement et service créés, le service est exposé sur le port `31496` dans le groupe de sécurité du noeud maître à accéder à ce service.



```
File Edit View Search Terminal Help
A s3 bucket is required to store cluster state information.
root@DevImranOps:~# kops update cluster kubernetes.devimranops.club --yes --state=s3://kops-state-86
I0330 18:12:19.022553    2871 executor.go:91] Tasks: 0 done / 56 total; 27 can run
I0330 18:12:20.430752    2871 vfs_castore.go:422] Issuing new certificate: "kubecfg"
I0330 18:12:20.590799    2871 vfs_castore.go:422] Issuing new certificate: "master"
I0330 18:12:20.771600    2871 vfs_castore.go:422] Issuing new certificate: "kubelet"
I0330 18:12:26.910766    2871 executor.go:91] Tasks: 27 done / 56 total; 12 can run
I0330 18:12:29.447253    2871 executor.go:91] Tasks: 39 done / 56 total; 15 can run
I0330 18:12:33.262695    2871 launchconfiguration.go:310] waiting for IAM instance profile "masters.kubernet
es.devimranops.club" to be ready
I0330 18:12:33.263549    2871 launchconfiguration.go:310] waiting for IAM instance profile "nodes.kubernetes
.devimranops.club" to be ready
I0330 18:12:45.146373    2871 executor.go:91] Tasks: 54 done / 56 total; 2 can run
I0330 18:12:46.170759    2871 executor.go:91] Tasks: 56 done / 56 total; 0 can run
I0330 18:12:46.170816    2871 dns.go:141] Pre-creating DNS records
I0330 18:12:50.787335    2871 update_cluster.go:208] Exporting kubecfg for cluster
Kops has set your kubectl context to kubernetes.devimranops.club

Cluster is starting.  It should be ready in a few minutes.

Suggestions:
 * validate cluster: kops validate cluster
 * list nodes: kubectl get nodes --show-labels
 * ssh to the master: ssh -i ~/.ssh/id_rsa admin@api.kubernetes.devimranops.club
 * read about installing addons: https://github.com/kubernetes/kops/blob/master/docs/addons.md

root@DevImranOps:~#
root@DevImranOps:~#
root@DevImranOps:~#
root@DevImranOps:~#
```

Figure 22: Alt Text

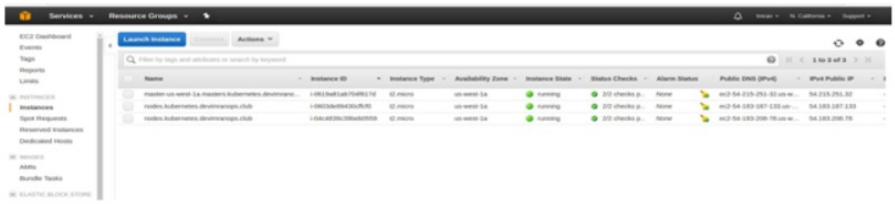


Figure 23: Alt Text

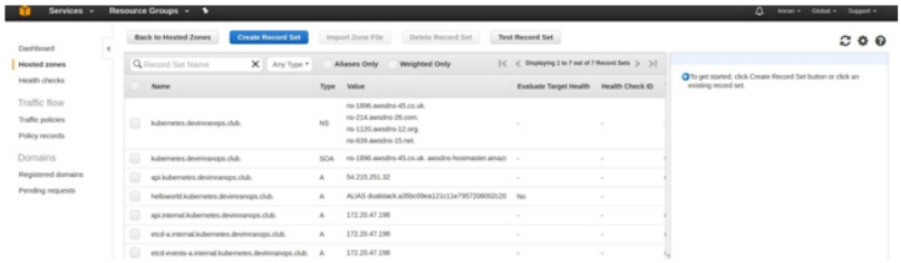
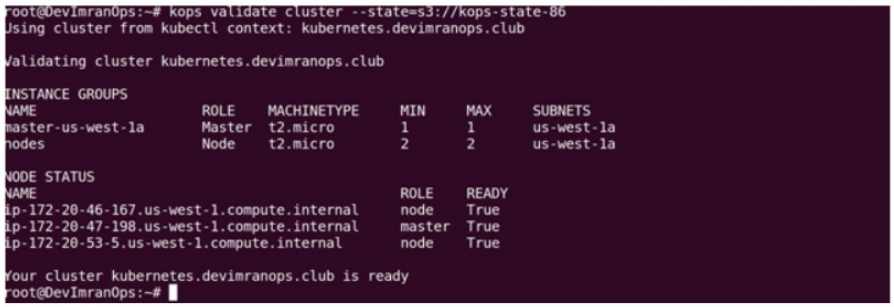
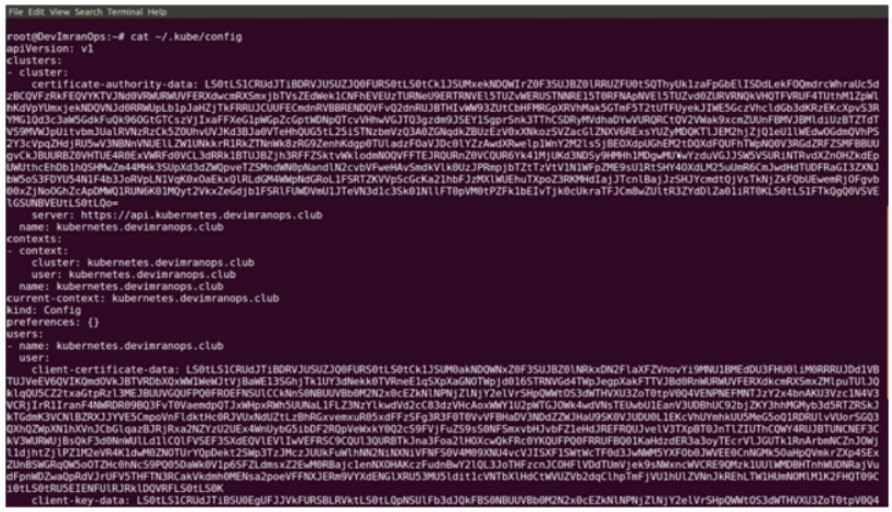


Figure 24: Alt Text



```
root@DevImranOps:~# kops validate cluster --state=s3://kops-state-86
Using cluster from kubectl context: kubernetes.devimranops.club

Validating cluster kubernetes.devimranops.club

INSTANCE GROUPS
NAME                    ROLE    MACHINETYPE     MIN     MAX     SUBNETS
master-us-west-1a       Master  t2.micro        1       1       us-west-1a
nodes                   Node    t2.micro        2       2       us-west-1a

NODE STATUS
NAME                                            ROLE    READY
ip-172-20-46-167.us-west-1.compute.internal     node    True
ip-172-20-47-198.us-west-1.compute.internal     master  True
ip-172-20-53-5.us-west-1.compute.internal       node    True

Your cluster kubernetes.devimranops.club is ready
root@DevImranOps:~#
```

Figure 25: Alt Text

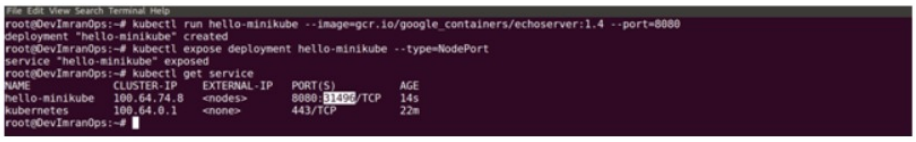


```
File  Edit  View  Search  Terminal  Help

root@DevImranOps:~# cat ~/.kube/config
apiVersion: v1
clusters:
- cluster:
    certificate-authority-data: LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0tCk1JSUMxekNDQWIrZ0F3SUJBZ0lRRUZFU0tSQThyUk1zaFpGbElISDdLekFOQmdrcWhraUc5d
zBCQVFzRkFEQVYKTVJNd0VRWURWUVFERXdwcmRXSmxjbTVsZEdWek1CNFhEVEUzTURNeU9ERTRNVEl5TUZvWERUSTNNRE15T0RFNApNVEl5TUZvd0ZURVRNQkVHQTFVRUF4TUthM1ZpWl
hKdVpYUmxjekNDQVNJd0RRWUpLb1pJaHZjTkFRRUJCUUFECmdnRVBBRENDQVFvQ2dnRUJBTHIvWW93ZUtCbHFMRGpXRVhMak5GTmF5T2tUTFUyekJIWE5GczVhcldGb3dKRzEKcXpv5JR
YMG1Qd3c3aW5GdkFuQk96OGtGTCszVjIxaFFXeGipWGpZcGptWDNpQTcvVHhwVGJTQ3gzdm9J5EY1SgprSnk3TThCSDRyMVdhaDYwVURQRCtQV2VWak9xcmZUUnFBMVJBMldiUzBTZTdT
NS9MYWJpUitvbmJUalRVNzRzCk5ZOUhvVVJKd3BJaRVTeHhQUG5tL25iSTNzbmVzQ3A0ZGNqdkZBUzEzV0xXNkozSVZacGlZNXV6RExsYUZyMDQKTlJEM2hjZjQ1eUl1WEdwOGdmQVhPS
2Y3cVpqZHdjRU5wV3NBNnVNUElLZW1UNkkrR1RkZTNnWkBzRG9ZenhKdgp0TUladzFOaVJDc0lYZzAwdXRwelp1WnY2M2lsSjBEOXdpUGhEM2tDQXdFQUFhTWpNQ0V3RGdZRFZSMFBBUU
gvCkJBUURBZ0VHTUE4R0ExVWRFd0VCL3dRRk1BTUJBZjh3RFFZSktvWklodmNOQVFFTEJRQURnZ0VCQUR6Yk41MjUKd3NDSy9HMHh1MDgwMUWwYzduVGJJSW5VSURiNTRvdXZnOHZkdEp
UWUthcEhDb1hQSHMwZm44MHk3SUpXd3dZWQpveTZSMndWNQpNandlN2cvbVFweHAvSmdkVlk0UzJPRmpjbTZtTzVtV1N1WFpZME9sU1RtSHY4QXdLM25uUmR6CmJwdHdTUDFRaGI3ZXNJ
bW5oS3FDYU54N1F4b3JoRVpLN1VqK0xOaEkxQlRLdGM4WWpNdGRoL1FSRTZKVVp5cGcKa21hbFJzMXlWUEhuTXpoZ3RKMHdIajJTcnlBajJzSHJYcmdtQjVsTkNjZkFQbUEwemRjOFgvb
00xZjNoOGhZcApOMWQ1RUN6K01MQytZVkxZeGdjb1FSRlFUWDVmU1JTeVN3d1c3Sk01NllfT0pVM0tPZFk1bEIvTjk0cUkraTFJCm8wZUltR3ZYdDlZa01iRT0KLS0tLS1FTkQgQ0VSVE
lGSUNBVEUtLS0tLQo=
    server: https://api.kubernetes.devimranops.club
  name: kubernetes.devimranops.club
contexts:
- context:
    cluster: kubernetes.devimranops.club
    user: kubernetes.devimranops.club
  name: kubernetes.devimranops.club
current-context: kubernetes.devimranops.club
kind: Config
preferences: {}
users:
- name: kubernetes.devimranops.club
  user:
    client-certificate-data: LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0tCk1JSUM0akNDQWNxZ0F3SUJBZ0lNRkxDN2FlaXFZVnovYi9MNU1BMEdDU3FHU0liM0RRRUJDd1VB
TUJVeEV6QVIKQmdOVkJBTVRDbXQxWW1WeWJtVjBaWE13SGhjTk1UY3dNekk0TVRneE1qSXpXaGN0TWpjd016STRNVGd4TWpJegpXakFTTVJBd0RnWURWUVFERXdkcmRXSmxZMlpuTUlJQ
klqQUSCZ2txaGtpRzl3MEJBUUVGQUFPQ0FROEFNSUlCCkNnS0NBUUVBb0M2N2x0cEZkNlNPNjZlNjY2elVrSHpQWWtOS3dWTHVXU3ZoT0tpV0Q4VENPNEFMNTJzY2x4bnAKU3Vzc1N4V3
VCRjIrR1IranF4NWRDR09BQ3FvT0VaemdpQTJxWHpxRWh5UUNaL1FLZ3NzYlkwdVd2cC83dzVHcAoxWWY1U2pWTGJOWk4wdVNsTEUwbU1EanY3UDBhUC92bjZKY3hhMGMyb3d5RTZRSkJ
kTGdmK3VCNlBZRXJJYVE5CmpoVnFldktHc0RJVUxNdUZtLzBhRGxvemxuR05xdFFzSFg3R3F0T0VvVFBHaDV3NDdZZWJHaU9SK0VJUDU0L1EKcVhUYmhkUU5MeG5oQ1RDRUlvVUorSGQ3
QXhQZWpXN1hXVnJCbGlqazBJRjRxaZNZYzU2UEx4WnUybG5ibDF2RQpVeWxkY0Q2cS9FVjFuZS9sS0NFSmxvbHJvbFZ1eHdJREFRQUJvelV3TXp8T0JnTlZIUThCQWY4RUJBTUNCNEF3C
kV3WURWDjBsQkF3d0NmWUlLdlCQlFVSEF3SXdEQVlEVlkvEFRSC9CQUI3QURBTkJna3Foa2lHOXcwQkFRcBYKQUFPQ0FRRUFBQQ1KaHdzdER3a3oyTEcrVLJGUTk1RnArbmNCZnJOWj
lldjhtZjlPZ1M2eVR4K1dwM0ZNOTUrYQpDekt2SWp3TzJMczJUUXFuWlhNN2N1NXNiVFNFS0V4M09XNU4vcVJISXFlSWtWcTF0d3JwWWM5YXF0b0JWVEE0CnNGMk5OaHpQVmkrZXp4SEx
ZUnBSWGRqQW5oOTZHc0hNcS9PQ05DaWk0V1p6SFZLdmsxZ2EwM0RBajc1enNXOHAKczFudnBwY2lQL3JoTHFzcnJCOHFlVDdTUmVjek9sNWxncWVCRE9QMzk1UUlWMDBHTnhWUDNRajVu
dFpnWDZwaQpRdVJrUFV5THFTN3RCakVkdmh0MENsa2poeVFFNXJERm9VYXdENGlXRU53MU5ldit1cVNTbXlHdCtWVUZVb2dqClhpTmFjVU1hUlZVNnJkREhLTW1HUmNOMlM1K2FHQT09C
i0tLS0tRU5EIENFUlRJRklDQVRFLS0tLS0K
    client-key-data: LS0tLS1CRUdJTiBSU0EgUFJJVkFURSBLRVktLS0tLQpNSUlFb3dJQkFBS0NBUUVBb0M2N2x0cEZkNlNPNjZlNjY2elVrSHpQWWtOS3dWTHVXU3ZoT0tpV0Q4
```

Figure 26: Alt Text

```
root@DevImranOps:~# kubectl get nodes
NAME                                            STATUS         AGE
ip-172-20-46-167.us-west-1.compute.internal     Ready          14m
ip-172-20-47-198.us-west-1.compute.internal     Ready,master   15m
ip-172-20-53-5.us-west-1.compute.internal       Ready          14m
root@DevImranOps:~#
```

Figure 27: Alt Text



```
File  Edit  View  Search  Terminal  Help
root@DevImranOps:~# kubectl run hello-minikube --image=gcr.io/google_containers/echoserver:1.4 --port=8080
deployment "hello-minikube" created
root@DevImranOps:~# kubectl expose deployment hello-minikube --type=NodePort
service "hello-minikube" exposed
root@DevImranOps:~# kubectl get service
NAME             CLUSTER-IP    EXTERNAL-IP   PORT(S)          AGE
hello-minikube   100.64.74.8   <nodes>       8080:31496/TCP   14s
kubernetes       100.64.0.1    <none>        443/TCP          22m
root@DevImranOps:~#
```

Figure 28: Alt Text

- Accédez à `AWS EC2` => Sélectionnez le nœud maître => Groupe de sécurité => `masters.kubernetes.Inbound` => Edit.
- Ajoutez le numéro de port et la source depuis n'importe où/MyIP.

Figure 29: Alt Text

- Obtenez l'adresse `IP publique` du noeud maître et accédez-y à partir du navigateur sur le port exposé.

Figure 30: Alt Text

- Au lieu de l'adresse `IP` publique du noeud maître, nous pouvons également utiliser les noms attribués par `Route53`.

Figure 31: Alt Text

- Si vous n'utilisez pas le cluster, vous le supprimer.

Conteneurs et images

- Dans cet exercice, nous allons créer une `image Docker` pour l'application `nodejs` qui utilisera `dockerhub` comme registre et exécutera cette image sur notre cluster `Kubernetes`.
- Vérifie le repertoire `docker-demo`. Vérifiez les trois fichiers ci-dessous "

Figure 32: Alt Text

```
Dockerfile
=========

# cat Dockerfile
FROM node:4.6
WORKDIR /app
ADD ./app
RUN npm install
EXPOSE 3000
CMD npm start

Index.JS
===========

# cat index.js

var express = require('express')
var app = express();
app.get('/', function(req, res) {
  res.send('Hello World!');
}

var server = app.listen(3000, function () {
  var host = server.address().address;
  var port = server.address().port;
  console.log('Example app listening at http://%:%s', host, port);
})


Package.Json
============

# cat package.json
{
  "name" : "myapp",
  "version" : "0.0.1",
  "private": true,
  "scripts": {
    "start": "node index.js"
  };
```

```
  "engines" : {
    "node" : "^4.6.1"
  };

  "dependencies" : {
      "express": "^4.14.0",
      "mysql" : "^2.10.2"
  }
}
```

- Installez Docker.

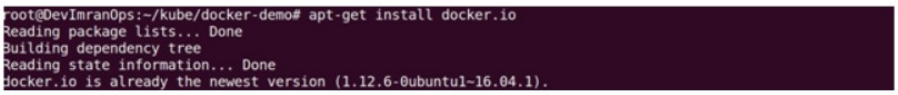


Figure 33: Alt Text

- Créer une image
  - Le nom des images doit correspondre à votre compte/nom de dépôt `Dockerhub`.
  - Par exemple, mon nom de compte dans `dockerhub est <<visualpath>>`, donc le nom de mon image est `visualpath/k8s-demo`.

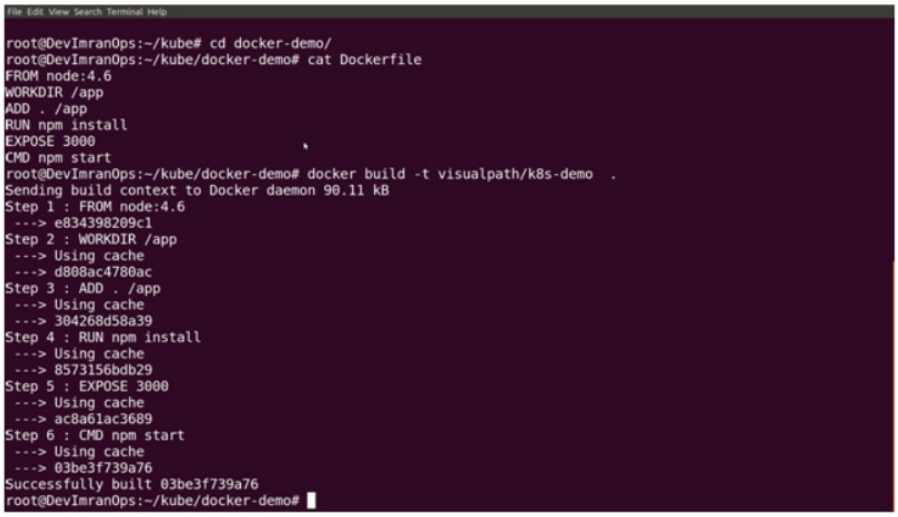


Figure 34: Alt Text

- Exécutez l'image localement et testez-la

Figure 35: Alt Text

- Testez-le depuis le navigateur
  - Entrez l'adresse `IP` de votre machine virtuelle `(kops vm)` sur laquelle votre conteneur est exécuté et le numéro de port.

Figure 36: Alt Text

- Transférez l'image vers `DockerHub`

Première application sur Kubernetes

- Exécutons notre nouvelle application sur le cluster `Kubernetes`
- Avant de pouvoir lancer un conteneur, nous devons créer une définition `POD`.
- Un `POD` décrit une application exécutée sur `Kubernetes`
- Un peut contenir un ou plusieurs conteneurs étroitement couplés qui constituent l'application.
  - Ces applications peuvent facilement communiquer entre elles en utilisant leurs numéros de port locaux.
  - Notre application n'a qu'un seul conteneur.
  - Lire la définition du `pod`.
- Utiliser `kubectl` pour créer un `pod` sur le cluster.
- Commandes `kubectl` utiles
- Exécuter des commandes sur le conteneur

```
File Edit View Search Terminal Tabs Help
root@DevImranOps: ~/kube/docker-demo        imran@DevOps: ~/.../kops
 ---> ac8a61ac3689
Step 6 : CMD npm start
 ---> Using cache
 ---> 03be3f739a76
Successfully built 03be3f739a76
root@DevImranOps:~/kube/docker-demo# docker images
REPOSITORY              TAG                 IMAGE ID            CREATED             SIZE
samnode                 latest              ebe9d473e708        13 minutes ago      649.9 MB
visualpath/docker-demo  latest              03be3f739a76        6 days ago          651.2 MB
visualpath/k8s-demo     latest              03be3f739a76        6 days ago          651.2 MB
node                    4.6                 e834398209c1        4 months ago        645.9 MB
root@DevImranOps:~/kube/docker-demo# docker login
Login with your Docker ID to push and pull images from Docker Hub. If you don't have a Docker ID, head over
to https://hub.docker.com to create one.
Username (visualpath):
Password:
Login Succeeded
root@DevImranOps:~/kube/docker-demo# docker push visualpath/k8s-demo
The push refers to a repository [docker.io/visualpath/k8s-demo]
8aac3e0fe990: Pushed
0160dcf30186: Pushed
2b11498abede: Pushed
e1da644611ce: Pushed
d79093d63949: Pushed
87cbe568afdd: Pushed
787c930753b4: Pushing 194.4 MB/318.3 MB
9f17712cba0b: Pushing 126.6 MB
223c0d04a137: Pushed
fe4c16cbf7a4: Pushing 128.9 MB
```

Figure 37: Alt Text

```
root@DevImranOps:~/kube# cat first-app/helloworld.yml
apiVersion: v1
kind: Pod
metadata:
  name: nodehelloworld.example.com
  labels:
    app: helloworld
spec:
  containers:
  - name: k8s-demo
    image: visualpath/k8s-demo
    ports:
    - name: nodejs-port
      containerPort: 3000
root@DevImranOps:~/kube#
```

Figure 38: Alt Text

```
root@DevImranOps:~/kube# kubectl create -f first-app/helloworld.yml
pod "nodehelloworld.example.com" created
root@DevImranOps:~/kube# kubectl get pod
NAME                        READY     STATUS    RESTARTS   AGE
nodehelloworld.example.com  1/1       Running   0          3m
```

```
root@DevImranOps:~/kube# kubectl describe pod nodehelloworld.example.com
Name:           nodehelloworld.example.com
Namespace:      default
Node:           ip-172-20-34-251.us-west-1.compute.internal/172.20.34.251
Start Time:     Sat, 01 Apr 2017 19:18:20 +0000
Labels:         app=helloworld
Status:         Running
IP:             100.96.2.2
Controllers:    <none>
```

Figure 39: Alt Text

Command	Description
kubectl get pod	Get information about all running pods
kubectl describe pod <pod>	Describe one pod
kubectl expose pod <pod> --port=444 --name=frontend	Expose the port of a pod (creates a new service)
kubectl port-forward <pod> 8080	Port forward the exposed pod port to your local machine
kubectl attach <podname> -i	Attach to the pod
kubectl exec <pod> -- command	Execute a command on the pod
kubectl label pods <pod> mylabel=awesome	Add a new label to a pod
kubectl run -i --tty busybox --image=busybox --restart=Never -- sh	Run a shell in a pod - very useful for debugging

Figure 40: Alt Text

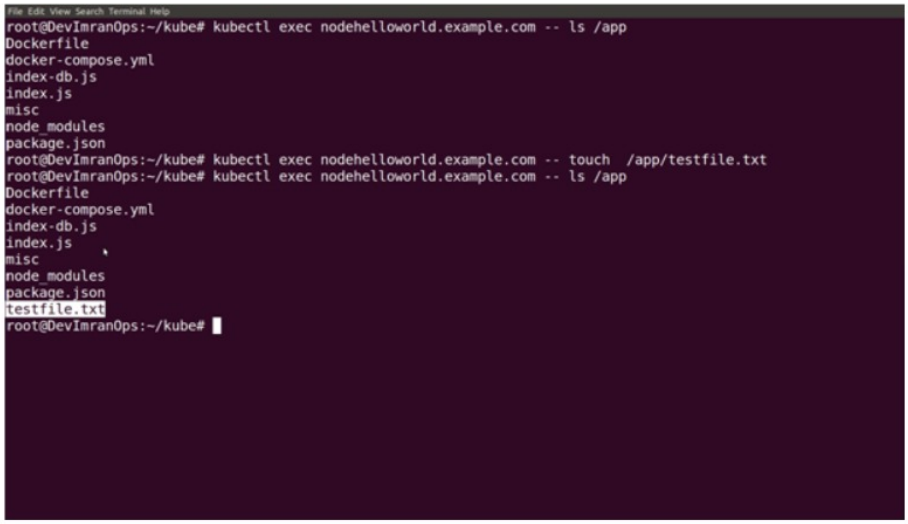


Figure 41: Alt Text

Presentation de Service

- Création de service pour notre application.

```
File Edit View Search Terminal Help
root@DevImranOps:~/kube# cat first-app/helloworld-nodeport-service.yml
apiVersion: v1
kind: Service
metadata:
  name: helloworld-service
spec:
  ports:
  - port: 31001
    nodePort: 31001
    targetPort: nodejs-port
    protocol: TCP
  selector:
    app: helloworld
  type: NodePort
root@DevImranOps:~/kube# kubectl create -f first-app/helloworld-nodeport-service.yml
service "helloworld-service" created
root@DevImranOps:~/kube# kubectl get service
NAME                 CLUSTER-IP    EXTERNAL-IP   PORT(S)           AGE
helloworld-service   100.68.5.83   <nodes>       31001:31001/TCP   5s
kubernetes           100.64.0.1    <none>        443/TCP           35m
root@DevImranOps:~/kube#
```

Figure 42: Alt Text

- Accès au service du monde extérieur.
- Ouvrez le groupe `MasterNode Security` depuis `AWS`.
- Autorisez le port `31001` depuis `MyIP/Anywhere`. Le port peut être différent,veuillez vérifier votre port de sevice (`kubectl get service`).
- Ouvrez le navigateur, entrez l'adresse `IP publique` de votre noeud maître et le port `31001`.
- Suppression d'un service
- Création d' `AWS ELB` en tant que service pour notre `pod`.
- Vérifiez `ELB` depuis `AWS`
- Attribuez un nom `DNS` à notre `ELB` à partir de `Route53`.
- Accédez au tableau de bord `AWS Route53` => `Zone hébergée` => Créer un jeu d'enregistrements => Nom: helloworld => Alias : Oui => Equilibreur de charge `ELB Classic`.
- Sélectionnez votre `ELB` et utilisez `helloworld.kubernetes.devimranops.club`(domaine différent dans votre cas).

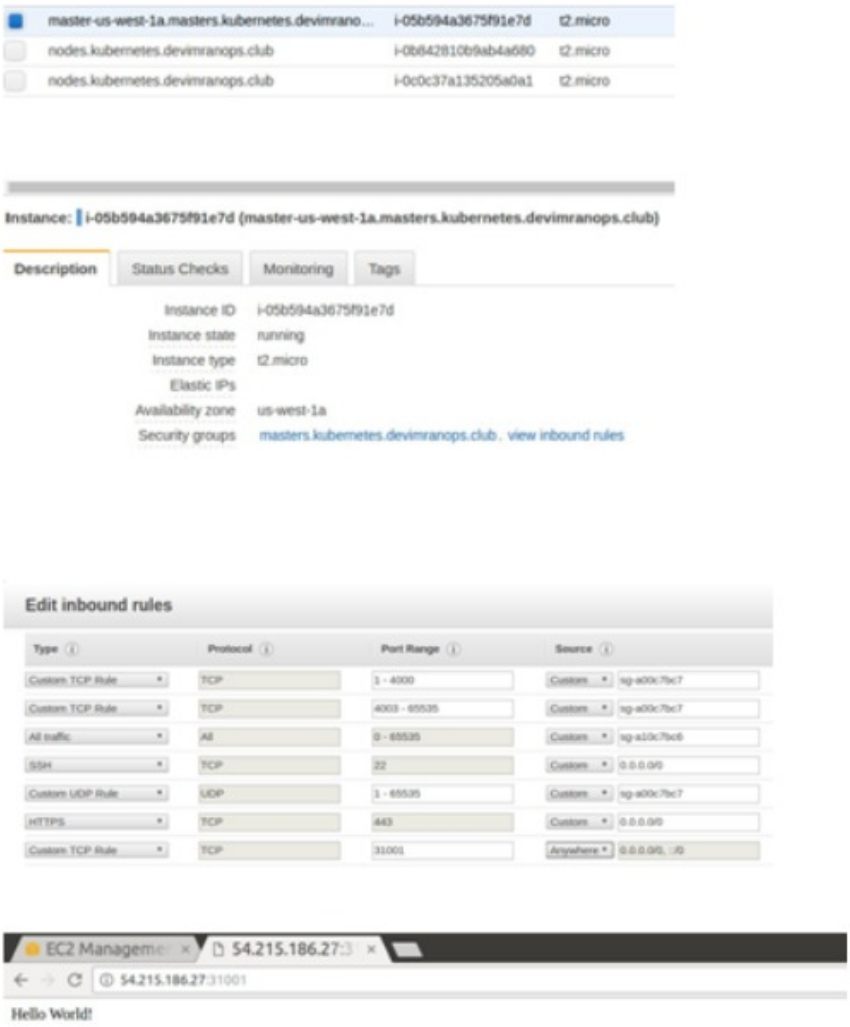


Figure 43: Alt Text

```
root@DevImranOps:~/kube# kubectl get svc
NAME                 CLUSTER-IP    EXTERNAL-IP   PORT(S)           AGE
helloworld-service   100.68.5.83   <nodes>       31001:31001/TCP   14m
kubernetes           100.64.0.1    <none>        443/TCP           49m
root@DevImranOps:~/kube# kubectl delete service helloworld-service
service "helloworld-service" deleted
root@DevImranOps:~/kube#
```

Figure 44: Alt Text

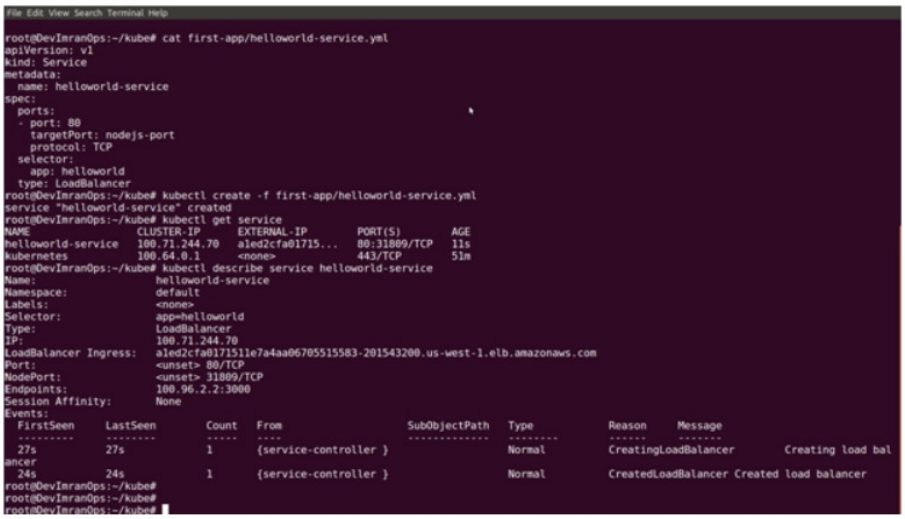


Figure 45: Alt Text

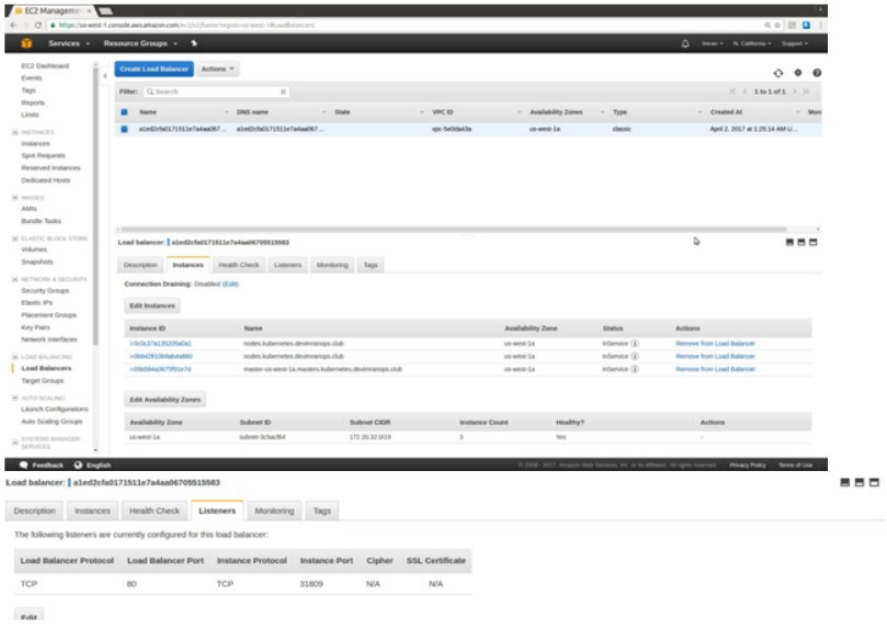
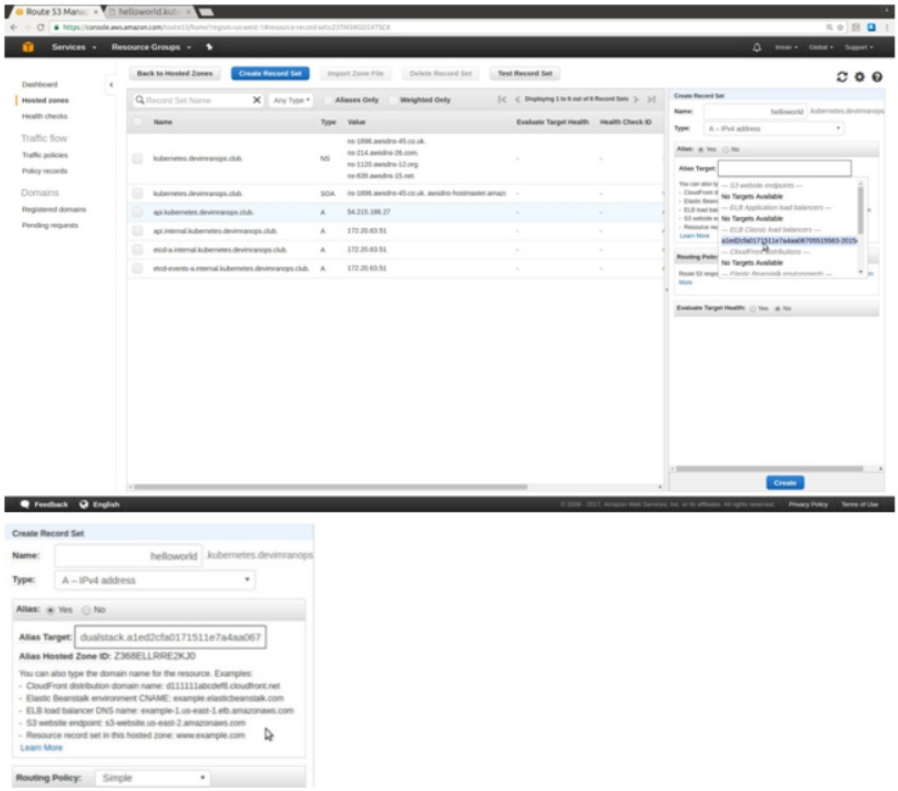


Figure 46: Alt Text



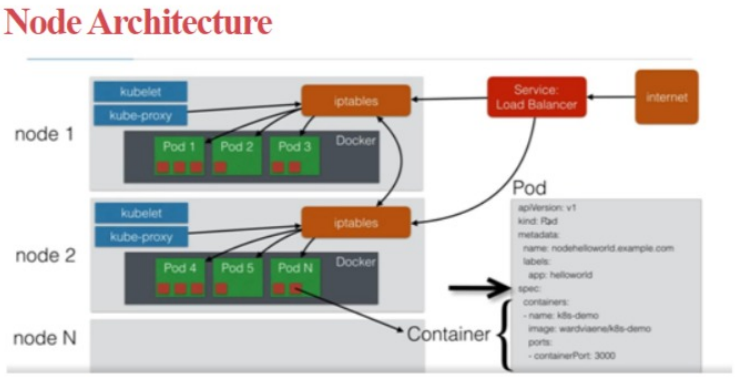


Figure 47: Alt Text

Mise à l'échelle des pods

Mise à l'échelle

- Si votre application est sans état, vous pouvez la mettre à l'échelle horizontalement.
- Sans état signifie que votre application ne stocke pas de fichiers/données sur le système de fichiers local.
- Toutes les bases de donnés sont avec état. Ils stockent les fichiers de base de données localement.
- La plupart des applications `Web` peuvent être rendues sans état.
- La gestion des sessions doît être effectuée en dehors du conteneur.
- Tout fichier qui doit être enregistré ne peut pas être enregistré localement sur l'ordinateur.
- La mise à l'échelle dans `Kubernetes` peut être effectuée à l'aide du `TReplication Controller`.
- Le contrôleur de réplication garantira qu'un nombre spécifié de répliques de `pods` s'exécuteront à tout moment.
- Un `pod` créé avec le contrôleur de réplication sera automatiquement remplacé s'il est rempli, supprimé ou terminé.
- L'utilisation du contrôleur de replication est également recommandée si vous souhaitez simplement vous assurer qu'un `pod` est toujours en cours d'exécution, même après les redémarrages.
- Répliquer notre `pod` deux fois.

```
root@DevImranOps:~/kube# cat replication-controller/helloworld-repl-controller.yml
apiVersion: v1
kind: ReplicationController
metadata:
  name: helloworld-controller
spec:
  replicas: 2
  selector:
    app: helloworld
  template:
    metadata:
      labels:
        app: helloworld
    spec:
      containers:
      - name: k8s-demo
        image: visualpath/k8s-demo
        ports:
        - name: nodejs-port
          containerPort: 3000
```

Figure 48: Alt Text

- Vérifiez `pod & rc` (contrôler de réplication) avec la commande `kubectl get & kubectlscribe`.
- Supprimez un `pod` pour voir la même à l'échelle.
- Augenter/réduire le contrôleur de réplication.
- Augmenter/Réduire avec le fichier de définition du contrôleur de réplication.

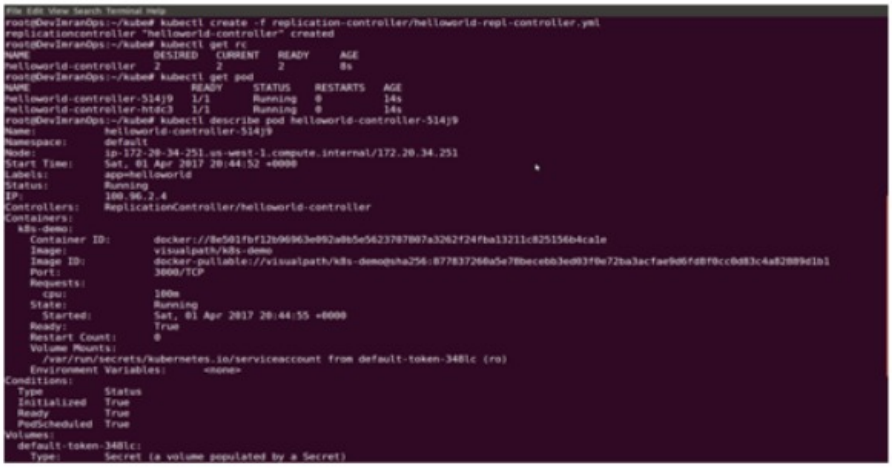


Figure 49: Alt Text

Figure 50: Alt Text

Figure 51: Alt Text

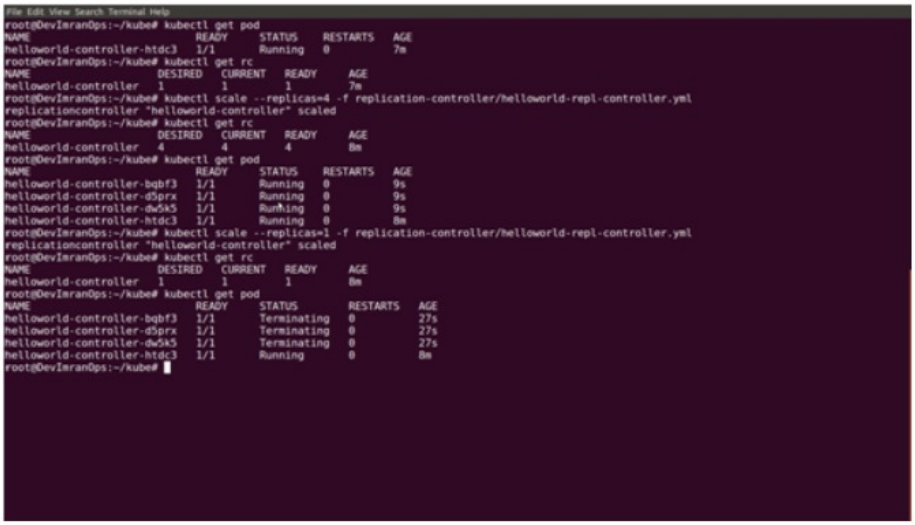


Figure 52: Alt Text

Déploiements

Ensemble de réplication

- Le jeu de réplicas est le `Replication Controller` de nouvelle génération.
- Il prend en charge un nouveau sélecteur capable d'effectuer une sélection basée sur un filtrage selon un ensemble de valeurs.
- Par exemple `environnement` soit `dev` ou `qa` non seulement basé sur l'égalité comme le `Replication Controller` -> par exemple `<<environnement>> == <<dev>>`.
- Cet ensemble de réplicas, plutôt que le `Replication Controller`, est utilisé par l'objet de déploiement.

Déploiements

- Une déclaration de déploiement dans `Kubernetes` vous permet d'effectuer des déploiements et des mises à jour d'applications.
- Lorsque vous utilisez l'objet de déploiement, vous définissez l'état de votre application.
- `Kubernetes` fera alors en sorte que les objets que les clusters correspondent à l'état souhaité.
- La simple utilisation du contrôleur de réplication ou du jeu de réplication peut s'avérer fastidieuse pour déployer des applications.
- L'objet de déploiement est plus facile à utiliser et vous offre plus de

possibilités.

- Avec un objet de déploiement, vous pouvez :
  - Créer un déploiement (par exemple, déployer une application).
  - Mettre à jour un déploiement (par exemple, déployer une dernière version)
  - Effectuer des mises à jour progressives (déploiements sans temps d'arrêt)
  - Revenir à une version précédente.
  - Pause/Reprendre (par exemple pour un déploiement jusqu'à un certain pourcentage seulement).
- Exemple de définition de déploiement

```
root@DevImranOps:~/kube# cat deployment/helloworld.yml
apiVersion: extensions/v1beta1
kind: Deployment
metadata:
  name: helloworld-deployment
spec:
  replicas: 3
  template:
    metadata:
      labels:
        app: helloworld
    spec:
      containers:
      - name: k8s-demo
        image: visualpath/k8s-demo
        ports:
        - name: nodejs-port
          containerPort: 3000
root@DevImranOps:~/kube#
```

Figure 53: Alt Text

- Commandes utiles

Command	Description
kubectl get deployments	Get information on current deployments
kubectl get rs	Get information about the replica sets
kubectl get pods --show-labels	get pods, and also show labels attached to those pods
kubectl rollout status deployment/helloworld-deployment	Get deployment status
kubectl set image deployment/helloworld-deployment k8s-demo=k8s-demo:2	Run k8s-demo with the image label version 2
kubectl edit deployment/helloworld-deployment	Edit the deployment object
kubectl rollout status deployment/helloworld-deployment	Get the status of the rollout
kubectl rollout history deployment/helloworld-deployment	Get the rollout history
kubectl rollout undo deployment/helloworld-deployment	Rollback to previous version
kubectl rollout undo deployment/helloworld-deployment --to-revision=n	Rollback to any version version

Figure 54: Alt Text

- Déploiement :
  - Créer un déploiement
  - Obtenir des informations sur le déploiement, le jeu de réplication (rs), le pod, l'état du déploiement.

```
root@DevImranOps:~/kube# kubectl create -f deployment/helloworld.yml
deployment "helloworld-deployment" created
root@DevImranOps:~/kube# kubectl get deployment
NAME                   DESIRED   CURRENT   UP-TO-DATE   AVAILABLE   AGE
helloworld-deployment   3         3         3            0           27s
root@DevImranOps:~/kube# kubectl get rs
NAME                              DESIRED   CURRENT   READY     AGE
helloworld-deployment-706070162   3         3         3         37s
root@DevImranOps:~/kube# kubectl get pods
NAME                                   READY     STATUS    RESTARTS   AGE
helloworld-deployment-706070162-8n3bs   1/1       Running   0          46s
helloworld-deployment-706070162-nlpdl   1/1       Running   0          46s
helloworld-deployment-706070162-q350d   1/1       Running   0          46s
root@DevImranOps:~/kube# kubectl get pods --show-labels
NAME                                   READY     STATUS    RESTARTS   AGE       LABELS
helloworld-deployment-706070162-8n3bs   1/1       Running   0          59s       app=helloworld,pod-template-hash=706070162
helloworld-deployment-706070162-nlpdl   1/1       Running   0          59s       app=helloworld,pod-template-hash=706070162
helloworld-deployment-706070162-q350d   1/1       Running   0          59s       app=helloworld,pod-template-hash=706070162
root@DevImranOps:~/kube# kubectl rollout status deployment helloworld-deployment
deployment "helloworld-deployment" successfully rolled out
root@DevImranOps:~/kube#
```

Figure 55: Alt Text

- Créez l'image de démonstration `k8s` avec la prochaine version `V2`.
- Copiez le contenu du répertoire `docker-demo` dans un autre repertoire.
- Modifiez le fichier `index.js`, mettre à jour la chaîne dans `res.send` vers quelque chose comme `V2`.
- Construisez l'image avec la balise `V2`.
- Envoyez la dernière image vers le registre `DockerHub`.
- Vérifiez depuis `DockerHub`.
- Exposez le déploiement existant avec `NodePort`.
- `NodePort` est `32400` selon la sortie ci-dessus.
- Autorisez ce port dans le groupe de sécurité du noeud maître afin que nous puissions nous connecter à ce service.
- Vérifiez l'adresse `IP` du noeud maître est le port exposé.
- Mettez à niveau la version de l'image vers `V2` à partir du déploiement.
- Vérifiez depuis dans le navigateur et curl.
- Historique du déploiement.
- Restaurer la version précédente.
- Augmentation de la limite d'historique de déploiement de deux à plusieurs.
- Déploiement avec un numéro de version

En savoir plus sur les services

- Les `pods` sont très dynamiques, ils vont et viennent sur le `cluster Kubernetes`.

```
root@DevImranOps:~/kube/docker-demo# ls
docker-compose.yml  Dockerfile  index-db.js  index.js  misc  package.json
root@DevImranOps:~/kube/docker-demo# cp * -r /tmp/hellov2/
root@DevImranOps:~/kube/docker-demo# cd /tmp/hellov2/
root@DevImranOps:/tmp/hellov2# vi index.js
```

```
root@DevImranOps:/tmp/hellov2# cat index.js
var express = require('express');
var app = express();

app.get('/', function (req, res) {
  res.send('Hello Everybody, this is version 2');
});

var server = app.listen(3000, function () {
  var host = server.address().address;
  var port = server.address().port;

  console.log('Example app listening at http://%s:%s', host, port);
});
```

```
root@DevImranOps:/tmp/hellov2# docker build -t visualpath/k8s-demo:V2 .
Sending build context to Docker daemon 8.704 kB
Step 1 : FROM node:4.6
 ---> e834398209c1
Step 2 : WORKDIR /app
 ---> Using cache
 ---> d808ac4780ac
Step 3 : ADD . /app
 ---> 7f2f05cfe161
Removing intermediate container 356b8e9ec00a
Step 4 : RUN npm install
 ---> Running in 5f01e5862183
npm info it worked if it ends with ok
```

```
root@DevImranOps:/tmp/hellov2# docker images
REPOSITORY                  TAG                 IMAGE ID            CREATED             SIZE
visualpath/k8s-demo         V2                  39b2b75413a5        2 minutes ago       651.2 MB
<none>                      <none>              5418af191c46        7 minutes ago       651.2 MB
samnode                     latest              ebe9d473e708        11 hours ago        649.9 MB
visualpath/docker-demo      latest              03be3f739a76        6 days ago          651.2 MB
node                        4.6                 e834398209c1        4 months ago        645.9 MB
root@DevImranOps:/tmp/hellov2# docker push visualpath/k8s-demo:V2
The push refers to a repository [docker.io/visualpath/k8s-demo]
2987eec31172: Pushed
996cf1e370b0: Pushed
2b11498abede: Layer already exists
e1da644611ce: Layer already exists
d79093d63949: Layer already exists
87cbe568afdd: Layer already exists
787c930753b4: Layer already exists
9f17712cba0b: Layer already exists
223c0d04a137: Layer already exists
fe4c16cbf7a4: Layer already exists
V2: digest: sha256:8496f6a052ca8b43eb97a6aa48fa4b17a8386bec148ec628051611bd5e47d2c3 size: 2420
```

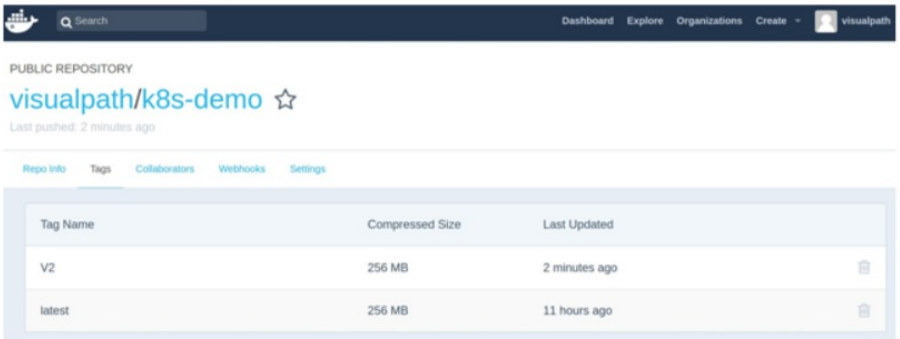


Figure 56: Alt Text

```
root@DevImranOps:~/kube# kubectl expose deployment helloworld-deployment --type=NodePort
service "helloworld-deployment" exposed
root@DevImranOps:~/kube# kubectl get service
NAME                   CLUSTER-IP       EXTERNAL-IP   PORT(S)          AGE
helloworld-deployment  100.70.171.200   <nodes>       3000:32400/TCP   10s
kubernetes             100.64.0.1       <none>        443/TCP          35m
root@DevImranOps:~/kube# kubectl describe service helloworld-deployment
Name:                   helloworld-deployment
Namespace:              default
Labels:                 app=helloworld
Selector:               app=helloworld
Type:                   NodePort
IP:                     100.70.171.200
Port:                   <unset> 3000/TCP
NodePort:               <unset> 32400/TCP
Endpoints:              100.96.1.6:3000,100.96.2.4:3000,100.96.2.5:3000
Session Affinity:       None
No events.
```

Figure 57: Alt Text

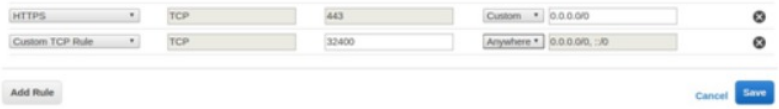


Figure 58: Alt Text

Figure 59: Alt Text

Figure 60: Alt Text

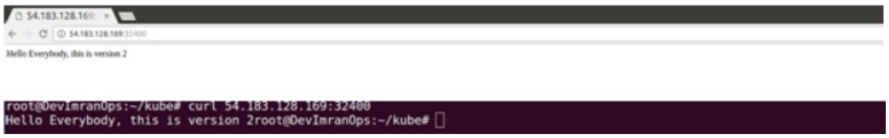


Figure 61: Alt Text

Figure 62: Alt Text

Figure 63: Alt Text

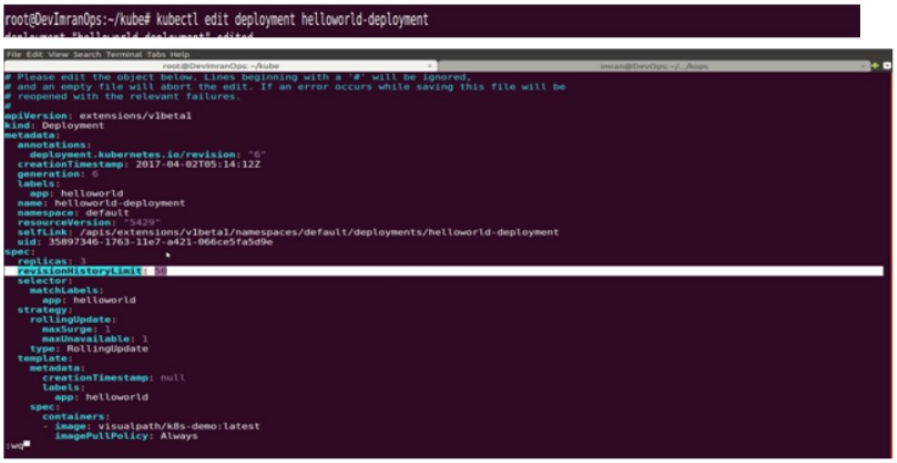


Figure 64: Alt Text

Figure 65: Alt Text

- Lors de l'utilisation d'un `Replication Controller`, les `pods` sont terminés et créés lors des opérations de mise à l'échelle.
- Lors de l'utilisation de déploiements, lors de la mise à jour de la version de l'image, les pods sont terminés et les nouveaux pods remplacent les anciens pods.
- C'est pourquoi les `Pods` ne doivent être jamais être accessibles directement, mais toujours via un service.
- Un `service` est le pont logique entre les pods `mortels` et d'autres services oiu utilisateurs finaux.
- Lors de l'utilisation précédente de la commande `kubectl expose`, vous avez créé un nouveau service pour votre `pod`, afin qu'il soit accessible en externe.
- La création d'un service créera un point de terminaison pour vos pods.
- Un `ClusterIP` : une adresse `IP` virtuelle accessible uniquement depuis l'intérieur du cluster.
- Un `NodePort`: un port identique sur chaque noeud et également accessible en externe.
- Un `LoadBalancer` : un `LoadBalancer` créé par le fournisseur de cloud qui acheminera le trafic externe vers chaque noeud du `NodePort`.
- Créez un module `helloworld`.

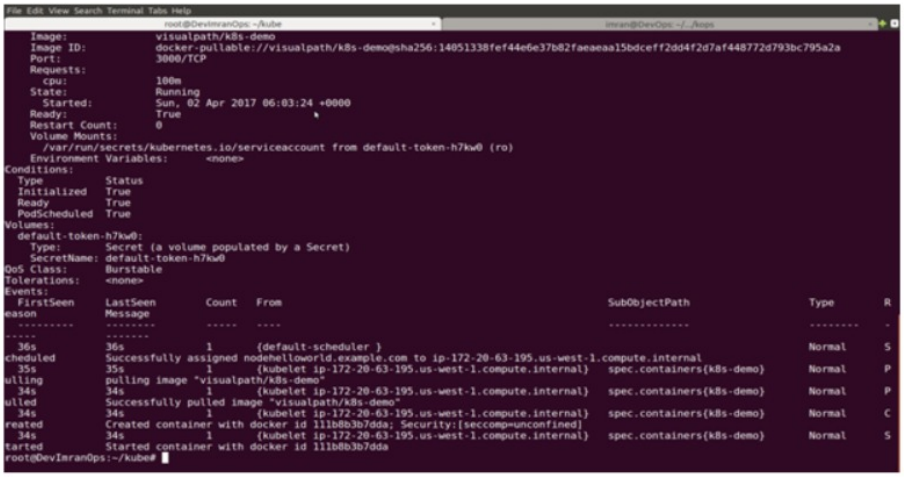


Figure 66: Alt Text

- Créez un service pour le `pod` ci-dessus, la valeur du sélecteur est le nom du `pod` tel qu'il est mis en évidence.

```
root@DevImranOps:~/kube# cat first-app/helloworld-nodeport-service.yml
apiVersion: v1
kind: Service
metadata:
  name: helloworld-service
spec:
  ports:
  - port: 31001
    nodePort: 31001
    targetPort: nodejs-port
    protocol: TCP
  selector:
    app: helloworld
  type: NodePort
root@DevImranOps:~/kube# kubectl create -f first-app/helloworld-nodeport-service.yml
service "helloworld-service" created
root@DevImranOps:~/kube# kubectl get service
NAME                CLUSTER-IP      EXTERNAL-IP   PORT(S)           AGE
helloworld-service  100.67.119.123  <nodes>       31001:31001/TCP   2m
kubernetes          100.64.0.1      <none>        443/TCP           1h
```

Figure 67: Alt Text

- `NodePort` est explicitement défini dans la définition du service, observez la capture d'écran ci-dessus.

```
root@DevImranOps:~/kube# cat first-app/helloworld-nodeport-service.yml
apiVersion: v1
kind: Service
metadata:
  name: helloworld-service
spec:
  ports:
  - port: 31001
    nodePort: 31001
    targetPort: nodejs-port
    protocol: TCP
  selector:
    app: helloworld
  type: NodePort
```

Figure 68: Alt Text

- Nous pouvons utiliser `svc` comme forme abrégée pour service.

```
root@DevImranOps:~/kube# kubectl get svc
NAME                CLUSTER-IP      EXTERNAL-IP   PORT(S)           AGE
helloworld-service  100.67.119.123  <nodes>       31001:31001/TCP   5m
kubernetes          100.64.0.1      <none>        443/TCP           1h
root@DevImranOps:~/kube# kubectl describe svc helloworld-service
Name:                   helloworld-service
Namespace:              default
Labels:                 <none>
Selector:               app=helloworld
Type:                   NodePort
IP:                     100.67.119.123
Port:                   <unset> 31001/TCP
NodePort:               <unset> 31001/TCP
Endpoints:              100.96.2.21:3000
Session Affinity:       None
No events.
```

Figure 69: Alt Text

- Supprimer et recréer le service modifiera l'adresse `IP/IP` virtuelle du cluster interne.

Etiquettes

- Les étiquettes sont des paires clé/valeur qui peuvent être attachées aux objets.
- Les étiquettes sont comme les balises dans `AWS`.
- Vous pouvez étiqueter vos objets, par exemple votre pod, en suivant une structure organisationnelle.
- Clé : environnement _ Valeur : `dev/staging/qa/prod`

```
File Edit View Search Terminal Tabs Help
root@DevImranOps: ~/kube                    imran@DevOps: ~/../kops
root@DevImranOps:~/kube# kubectl describe svc helloworld-service
Name:                   helloworld-service
Namespace:              default
Labels:                 <none>
Selector:               app=helloworld
Type:                   NodePort
IP:                     100.67.119.123
Port:                   <unset> 31001/TCP
NodePort:               <unset> 31001/TCP
Endpoints:              100.96.2.21:3000
Session Affinity:       None
No events.
root@DevImranOps:~/kube# kubectl delete svc helloworld-service
service "helloworld-service" deleted
root@DevImranOps:~/kube# kubectl create -f first-app/helloworld-service.yml
service "helloworld-service" created
root@DevImranOps:~/kube# kubectl get svc
NAME                   CLUSTER-IP      EXTERNAL-IP        PORT(S)        AGE
helloworld-service     100.69.176.221  ac6d9af47176f...   80:32678/TCP   7s
kubernetes             100.64.0.1      <none>             443/TCP        2h
root@DevImranOps:~/kube# kubectl describe svc helloworld-service
Name:                   helloworld-service
Namespace:              default
Labels:                 <none>
Selector:               app=helloworld
Type:                   LoadBalancer
IP:                     100.69.176.221
LoadBalancer Ingress:   ac6d9af47176f11e7a421066ce5fa5d9-897774110.us-west-1.elb.amazonaws.com
Port:                   <unset> 80/TCP
NodePort:               <unset> 32678/TCP
Endpoints:              100.96.2.21:3000
Session Affinity:       None
Events:
  FirstSeen    LastSeen     Count  From                    SubObjectPath  Type     Reason                  Message
  ---------    --------     -----  ----                    -------------  --------  ------                  -------
  15s          15s          1      {service-controller }                  Normal   CreatingLoadBalancer    Creating load balance
r
  13s          13s          1      {service-controller }                  Normal   CreatedLoadBalancer     Created load balancer
root@DevImranOps:~/kube#
```

Figure 70: Alt Text

- Clé : département - Valeur : `ingénierie/finance/marketing`

```
root@DevImranOps:~/kube# cat first-app/helloworld.yml
apiVersion: v1
kind: Pod
metadata:
  name: nodehelloworld.example.com
  labels:
    app: helloworld
spec:
```

Figure 71: Alt Text

- Les étiquettes ne sont pas uniques et plusieurs peuvent être ajoutées à un seul objet.
- Une fois les étiquettes attachées à un objet, vous pouvez utiliser des filtres pour affiner les résultats. C'est ce qu'on appelle des `sélecteurs d'étiquttes`.
- A l'aide des `sélecteurs d'étiquettes`, vous pouvez utiliser matching expression pour faire correspondre les étiquettes.
- Par exemple, un `pod` particulier ne peut s'exécuter que sur un noeud étiqueté avec `<<environnement>> == <<développement>>`.
- Vous pouvez utiliser des étiquettes pour baliser les noeuds.
- Une fois les noeuds balisés, vous pouvez utiliser des sélecteurs d'étiquettes pour permettre aux pods de s'exécuter uniquement sur des noeuds spécifiques.

- Sélection d'un noeud spécifique à partir de la définition dedéploiement.

```
root@DevImranOps:~/kube# cat deployment/helloworld-nodeselector.yml
apiVersion: extensions/v1beta1
kind: Deployment
metadata:
  name: helloworld-deployment
spec:
  replicas: 3
  template:
    metadata:
      labels:
        app: helloworld
    spec:
      containers:
      - name: k8s-demo
        image: wardviaene/k8s-demo
        ports:
        - name: nodejs-port
          containerPort: 3000
      nodeSelector:
        hardware: high-spec
```

Figure 72: Alt Text

- `Tag Hardware` : la haute spécification n'est présente sur aucun noeud de notre cluster.

```
root@DevImranOps:~/kube# kubectl get nodes --show-labels
NAME                                         STATUS          AGE       LABELS
ip-172-20-44-214.us-west-1.compute.internal   Ready           2h        beta.kubernetes.io/arch=amd64,beta.kubernetes.io/instance-type=t2.micr
o,beta.kubernetes.io/os=linux,failure-domain.beta.kubernetes.io/region=us-west-1,failure-domain.beta.kubernetes.io/zone=us-west-1a,kubernetes
.io/hostname=ip-172-20-44-214.us-west-1.compute.internal
ip-172-20-58-217.us-west-1.compute.internal   Ready,master    2h        beta.kubernetes.io/arch=amd64,beta.kubernetes.io/instance-type=t2.micr
o,beta.kubernetes.io/os=linux,failure-domain.beta.kubernetes.io/region=us-west-1,failure-domain.beta.kubernetes.io/zone=us-west-1a,kubernetes
.io/hostname=ip-172-20-58-217.us-west-1.compute.internal,kubernetes.io/role=master
ip-172-20-63-195.us-west-1.compute.internal   Ready           2h        beta.kubernetes.io/arch=amd64,beta.kubernetes.io/instance-type=t2.micr
o,beta.kubernetes.io/os=linux,failure-domain.beta.kubernetes.io/region=us-west-1,failure-domain.beta.kubernetes.io/zone=us-west-1a,kubernetes
.io/hostname=ip-172-20-63-195.us-west-1.compute.internal
```

Figure 73: Alt Text

- Créez notre déploiement qui a la balise `nodeSelector` recherchant du matériel: balise `high-spec`.
- Observez que l'état de création du pod est en attente.
- Le déploiement recherche un noeud avec une balise `high-spec`, qui n'est pas trouvé.
- Si nous n'avons pas une telle balise, la création du `pod` sera en attente.
- Marquons un noeud avec du matériel : haute spécification.
- Le pod sera automatiquement programmé une fois que l'étiquette est disponible.

Secrets

- `Secrets` fournit un moyen dans `Kubernetes` de distribuer des informations d'identification, des clés et des mots de passe aux `pods`.
- `Kubernetes` lui-même utilise ce mécanisme de secrets pour fournir les informations d'identification permettant d'accéder à `L'API interne`.
- Les secrets peuvent être utilisés des manières suivantes.
  - Utilisez des secrets comme variables d'environnement.

```
root@DevImranOps:~/kube# cat deployment/helloworld-nodeselector.yml
apiVersion: extensions/v1beta1
kind: Deployment
metadata:
  name: helloworld-deployment
spec:
  replicas: 3
  template:
    metadata:
      labels:
        app: helloworld
    spec:
      containers:
      - name: k8s-demo
        image: visualpath/k8s-demo
        ports:
        - name: nodejs-port
          containerPort: 3000
      nodeSelector:
        hardware: high-spec
root@DevImranOps:~/kube# kubectl create -f deployment/helloworld-nodeselector.yml
deployment "helloworld-deployment" created
root@DevImranOps:~/kube# kubectl get pod
NAME                                      READY     STATUS    RESTARTS   AGE
helloworld-deployment-2524499403-6pb8z    0/1       Pending   0          9s
helloworld-deployment-2524499403-9c76j    0/1       Pending   0          9s
helloworld-deployment-2524499403-qz0t3    0/1       Pending   0          9s
```

```
File  Edit  View  Search  Terminal  Tabs  Help
root@DevImranOps: ~/kube                                  imran@DevOps: ~/.../kops
NAME                                      READY     STATUS    RESTARTS   AGE
helloworld-deployment-2524499403-6pb8z    0/1       Pending   0          9s
helloworld-deployment-2524499403-9c76j    0/1       Pending   0          9s
helloworld-deployment-2524499403-qz0t3    0/1       Pending   0          9s
nodehelloworld.example.com                1/1       Running   0          1h
root@DevImranOps:~/kube# kubectl describe pod helloworld-deployment-2524499403-6pb8z
Name:           helloworld-deployment-2524499403-6pb8z
Namespace:      default
Node:           /
Labels:         app=helloworld
                pod-template-hash=2524499403
Status:         Pending
IP:
Controllers:    ReplicaSet/helloworld-deployment-2524499403
Containers:
  k8s-demo:
    Image:      visualpath/k8s-demo
    Port:       3000/TCP
    Requests:
      cpu:      100m
    Volume Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from default-token-h7kw0 (ro)
    Environment Variables:      <none>
Conditions:
  Type          Status
  PodScheduled  False
Volumes:
  default-token-h7kw0:
    Type:       Secret (a volume populated by a Secret)
    SecretName: default-token-h7kw0
QoS Class:      Burstable
Tolerations:    <none>
Events:
  FirstSeen     LastSeen        Count   From                    SubObjectPath   Type            Reason                  Message
  ---------     --------        -----   ----                    -------------   --------        ------                  -------
  4m            5s              19      {default-scheduler }                    Warning         FailedScheduling        pod (helloworld-deplo
yment-2524499403-6pb8z) failed to fit in any node
fit failure summary on nodes : MatchNodeSelector (3), PodToleratesNodeTaints (1)
root@DevImranOps:~/kube#
```

Figure 74: Alt Text

```
root@DevImranOps:~/kube# kubectl get nodes
NAME                                        STATUS         AGE
ip-172-20-44-214.us-west-1.compute.internal   Ready          3h
ip-172-20-58-217.us-west-1.compute.internal   Ready,master   3h
ip-172-20-63-195.us-west-1.compute.internal   Ready          3h
root@DevImranOps:~/kube# kubectl label nodes ip-172-20-44-214.us-west-1.compute.internal hardware=high-spec
node "ip-172-20-44-214.us-west-1.compute.internal" labeled
```

```
root@DevImranOps:~/kube# kubectl get nodes --show-labels
NAME                                        STATUS         AGE       LABELS
ip-172-20-44-214.us-west-1.compute.internal   Ready          3h        beta.kubernetes.io/arch=amd64,beta.kubernetes.io/instance-type=t2.micr
o,beta.kubernetes.io/os=linux,failure-domain.beta.kubernetes.io/region=us-west-1,failure-domain.beta.kubernetes.io/zone=us-west-1a,hardware=h
igh-spec,kubernetes.io/hostname=ip-172-20-44-214.us-west-1.compute.internal
ip-172-20-58-217.us-west-1.compute.internal   Ready,master   3h        beta.kubernetes.io/arch=amd64,beta.kubernetes.io/instance-type=t2.micr
```

Figure 75: Alt Text

```
root@DevImranOps:~/kube# kubectl get pods
NAME                                      READY     STATUS    RESTARTS   AGE
helloworld-deployment-2524499403-6pb8z    1/1       Running   0          11m
helloworld-deployment-2524499403-9c76j    1/1       Running   0          11m
helloworld-deployment-2524499403-qz0t3    1/1       Running   0          11m
nodehelloworld.example.com                1/1       Running   0          1h
root@DevImranOps:~/kube#
```

```
File  Edit  View  Search  Terminal  Tabs  Help
root@DevImranOps: ~/kube                                  imran@DevOps: ~/.../kops
root@DevImranOps:~/kube# kubectl describe pod helloworld-deployment-2524499403-6pb8z
Name:           helloworld-deployment-2524499403-6pb8z
Namespace:      default
Node:           ip-172-20-44-214.us-west-1.compute.internal/172.20.44.214
Start Time:     Sun, 02 Apr 2017 07:44:19 +0000
Labels:         app=helloworld
                pod-template-hash=2524499403
Status:         Running
IP:             100.96.1.13
Controllers:    ReplicaSet/helloworld-deployment-2524499403
Containers:
  k8s-demo:
    Container ID:       docker://d23cadaade95e576e2693db75c8cd82cb31444006144326a54460fc3452c144b
    Image:              visualpath/k8s-demo
    Image ID:           docker-pullable://visualpath/k8s-demo@sha256:14051338fef44e6e37b82faeaeaa15bdceff2dd4f2d7af448772d793bc795a2a
    Port:               3000/TCP
    Requests:
      cpu:              100m
    State:              Running
      Started:          Sun, 02 Apr 2017 07:44:21 +0000
    Ready:              True
    Restart Count:      0
    Volume Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from default-token-h7kw0 (ro)
    Environment Variables:      <none>
Conditions:
  Type          Status
  Initialized   True
  Ready         True
  PodScheduled  True
Volumes:
  default-token-h7kw0:
    Type:       Secret (a volume populated by a Secret)
    SecretName: default-token-h7kw0
QoS Class:      Burstable
Tolerations:    <none>
Events:
  FirstSeen     LastSeen        Count   From                    SubObjectPath   Type            Reason                  Message
  ---------     --------        -----   ----                    -------------   --------        ------                  -------
  12m           5m              29      {default-scheduler }                    Warning         FailedScheduling        pod (helloworld-deployment-2524499403-6pb8z) failed to fit in an
y node
fit failure summary on nodes : MatchNodeSelector (3), PodToleratesNodeTaints (1)
  4m    4m      1       {default-scheduler }                                    Normal  Scheduled       Successfully assigned helloworld-deployment-2524
499403-6pb8z to ip-172-20-44-214.us-west-1.compute.internal
  4m    4m      1       {kubelet ip-172-20-44-214.us-west-1.compute.internal}   spec.containers{k8s-demo}       Normal  Pulling         pulling image "visualpath/k8s-demo"
  4m    4m      1       {kubelet ip-172-20-44-214.us-west-1.compute.internal}   spec.containers{k8s-demo}       Normal  Pulled          Successfully pulled image "visualpath/k8s-demo"
  4m    4m      1       {kubelet ip-172-20-44-214.us-west-1.compute.internal}   spec.containers{k8s-demo}       Normal  Created         Created container with docker id d23cadaade95; S
ecurity:[seccomp=unconfined]
  4m    4m      1       {kubelet ip-172-20-44-214.us-west-1.compute.internal}   spec.containers{k8s-demo}       Normal  Started         Started container with docker id d23cadaade95
```

Figure 76: Alt Text

- Utilisez les secrets sous forme de fichier dans un `pod`.
- Cette configuration utilise des volumes à monter dans un conteneur.
- Dans les `volumes`, vous avez des fichiers.
- Encodage `Base 64`, encode du nom d'utilisateur et du mot de passe.

```
root@DevImranOps:~/kube# echo -n "root" | base64
cm9vdA==
root@DevImranOps:~/kube# echo -n "password" | base64
cGFzc3dvcmQ=
root@DevImranOps:~/kube#
```

Figure 77: Alt Text

- Création de secrets
- Création d'un déploiement qui montre notre secret (db-secrets) en tant que volume.
- `describe` montre les volumes montés, notre `db-secret` mis en surbrillance est monté en tant que volume.
- Connectez-vous à l'un des pods et vérifiez le point de montage `/etc/creds` pour vérifier nos secrets de base de données.

Déploiement WordPress

- Création de secrets pour le déploiement de `WordPress`

```
root@DevImranOps:~/kube# cat deployment/helloworld-secrets.yml
apiVersion: v1
kind: Secret
metadata:
  name: db-secrets
type: Opaque
data:
  username: cm9vdA==
  password: cGFzc3dvcmQ=
root@DevImranOps:~/kube# kubectl create -f deployment/helloworld-secrets.yml
secret "db-secrets" created
```

Figure 78: Alt Text

```
File  Edit  View  Search  Terminal  Tabs  Help
root@DevImranOps: ~/kube                                   imran@DevOps: ~/.../kops
    Port:               3000/TCP
    Requests:
      cpu:              100m
    State:              Running
      Started:          Sun, 02 Apr 2017 09:25:06 +0000
    Ready:              True
    Restart Count:      0
    Volume Mounts:
      /etc/creds from cred-volume (ro)
      /var/run/secrets/kubernetes.io/serviceaccount from default-token-h7kw0 (ro)
    Environment Variables:      <none>
Conditions:
  Type          Status
  Initialized   True
  Ready         True
  PodScheduled  True
Volumes:
  cred-volume:
    Type:       Secret (a volume populated by a Secret)
    SecretName: db-secrets
  default-token-h7kw0:
    Type:       Secret (a volume populated by a Secret)
    SecretName: default-token-h7kw0
QoS Class:      Burstable
Tolerations:    <none>
Events:
  FirstSeen     LastSeen        Count   From                                            SubObjectPath                   Type            Reason  M
essage
  ---------     --------        -----   ----                                            ------------                    --------        ------  -
------
  25s           25s             1       {default-scheduler }                                                            Normal          Scheduled
Successfully assigned helloworld-deployment-1674349767-33nzc to ip-172-20-63-195.us-west-1.compute.internal
  25s           25s             1       {kubelet ip-172-20-63-195.us-west-1.compute.internal}   spec.containers{k8s-demo}       Normal          Pulling p
ulling image "visualpath/k8s-demo"
  23s           23s             1       {kubelet ip-172-20-63-195.us-west-1.compute.internal}   spec.containers{k8s-demo}       Normal          Pulled  S
uccessfully pulled image "visualpath/k8s-demo"
  23s           23s             1       {kubelet ip-172-20-63-195.us-west-1.compute.internal}   spec.containers{k8s-demo}       Normal          Created C
reated container with docker id a0a442912537; Security:[seccomp=unconfined]
  23s           23s             1       {kubelet ip-172-20-63-195.us-west-1.compute.internal}   spec.containers{k8s-demo}       Normal          Started S
tarted container with docker id a0a442912537
root@DevImranOps:~/kube#
```

Figure 79: Alt Text

```
root@DevImranOps:~/kube# kubectl get pod
kubectNAME                                     READY     STATUS    RESTARTS   AGE
helloworld-deployment-1674349767-33nzc   1/1       Running   0          3m
helloworld-deployment-1674349767-czmp8   1/1       Running   0          3m
helloworld-deployment-1674349767-sgx62   1/1       Running   0          3m
nodehelloworld.example.com               1/1       Running   0          3h
root@DevImranOps:~/kube# kubectl exec -i -t helloworld-deployment-1674349767-33nzc -- /bin/bash
root@helloworld-deployment-1674349767-33nzc:/app# ls /etc/creds/
..4984_02_04_09_25_04.703846137/ ..data/                password                        username
root@helloworld-deployment-1674349767-33nzc:/app# cat /etc/creds/username
rootroot@helloworld-deployment-1674349767-33nzc:/app# cat /etc/creds/password
passwordroot@helloworld-deployment-1674349767-33nzc:/app#
```

```
passwordroot@helloworld-deployment-1674349767-33nzc:/app# df -h
Filesystem      Size  Used Avail Use% Mounted on
overlay          19G  3.5G   15G  20% /
tmpfs           498M     0  498M   0% /dev
tmpfs           498M     0  498M   0% /sys/fs/cgroup
tmpfs           498M  8.0K  498M   1% /etc/creds
/dev/xvda1       19G  3.5G   15G  20% /etc/hosts
shm              64M     0   64M   0% /dev/shm
tmpfs           498M   12K  498M   1% /run/secrets/kubernetes.io/serviceaccount
```

Figure 80: Alt Text

```
root@DevImranOps:~/kube# cat wordpress/wordpress-secrets.yml
apiVersion: v1
kind: Secret
metadata:
  name: wordpress-secrets
type: Opaque
data:
  db-password: cGFzc3dvcmQ=
root@DevImranOps:~/kube# kubectl create -f wordpress/wordpress-secrets.yml
secret "wordpress-secrets" created
root@DevImranOps:~/kube#
```

Figure 81: Alt Text

- Création du déploiement de `WordPress`

```
File Edit View Search Terminal Tabs Help
root@DevImranOps: ~/kube                    imran@DevOps: ~/.../kops
root@DevImranOps:~/kube# cat wordpress/wordpress-single-deployment-no-volumes.yml
apiVersion: extensions/v1beta1
kind: Deployment
metadata:
  name: wordpress-deployment
spec:
  replicas: 1
  template:
    metadata:
      labels:
        app: wordpress
    spec:
      containers:
      - name: wordpress
        image: wordpress:4-php7.0
        ports:
        - name: http-port
          containerPort: 80
        env:
          - name: WORDPRESS_DB_PASSWORD
            valueFrom:
              secretKeyRef:
                name: wordpress-secrets
                key: db-password
          - name: WORDPRESS_DB_HOST
            value: 127.0.0.1
      - name: mysql
        image: mysql:5.7
        ports:
        - name: mysql-port
          containerPort: 3306
        env:
          - name: MYSQL_ROOT_PASSWORD
            valueFrom:
              secretKeyRef:
                name: wordpress-secrets
                key: db-password
root@DevImranOps:~/kube# kubectl create -f wordpress/wordpress-single-deployment-no-volumes.yml
deployment "wordpress-deployment" created
root@DevImranOps:~/kube#
root@DevImranOps:~/kube#
```

Figure 82: Alt Text

- Création d'un service pour `WordPress`
- Accéder à votre déploiement `WordPress` depuis le monde extérieur.
- Accédez à l'adresse `IP pUblique` de votre noeud maître sur le port `31001` comme indiqué ci-dessus dans le port de service.
- Le groupe de sécurité du noeud maître doit autoriser l'accès au port `31001` depuis `MyIP/Anywhere`.
- Les `pods` ou `conteneurs` sont apatrides, ils ne sauvegardent pas de données s'ils sont tués ou morts.
- Nous supprimerons notre `pod WordPress` en tant que démo pour montrer ce comportement.
- Le `pod` est recréé par l'ensemble de `réplication de déploiement` si nous le supprimons manuellement.

```
root@DevImranOps:~/kube# cat wordpress/wordpress-service.yml
apiVersion: v1
kind: Service
metadata:
  name: wordpress-service
spec:
  ports:
  - port: 31001
    nodePort: 31001
    targetPort: http-port
    protocol: TCP
  selector:
    app: wordpress
  type: NodePort
root@DevImranOps:~/kube# kubectl create -f wordpress/wordpress-service.yml
service "wordpress-service" created
root@DevImranOps:~/kube# kubectl get svc
NAME                 CLUSTER-IP     EXTERNAL-IP   PORT(S)           AGE
kubernetes           100.64.0.1     <none>        443/TCP           5h
wordpress-service    100.64.1.146   <nodes>       31001:31001/TCP   1m
root@DevImranOps:~/kube#
```

Figure 83: Alt Text

- Toutes les données du `pod` précédent sont supprimées au fur et à mesure que nous supprimons le `pod`.
- Vérifiez l'application `WordPress` depuis le navigateur, nous aurons perdu l'installation précédente.

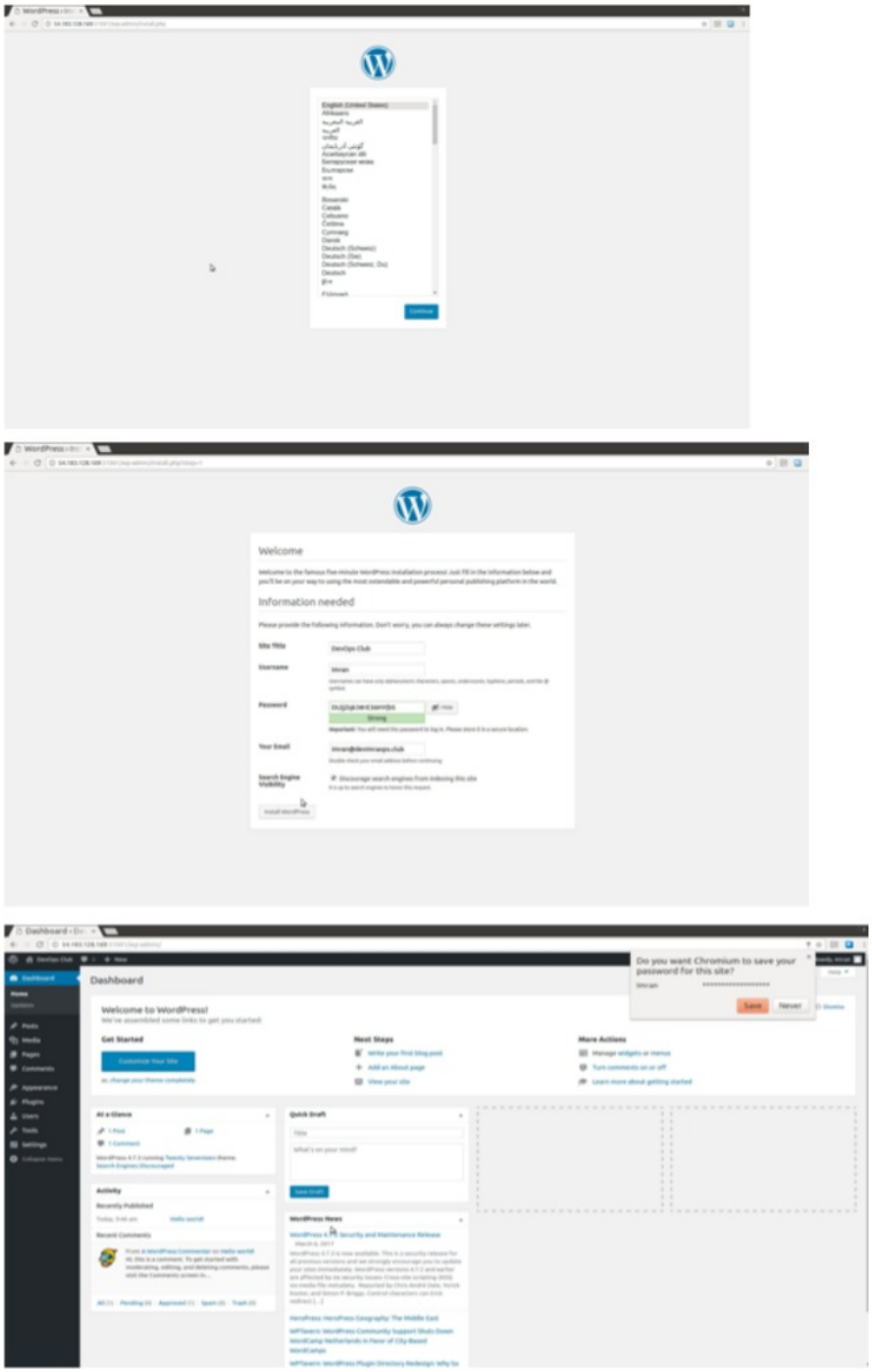


Figure 84: Alt Text

```
root@DevImranOps:~/kube# kubectl get pod
NAME                                    READY     STATUS    RESTARTS   AGE
wordpress-deployment-1969810892-z76tv   2/2       Running   0          16m
root@DevImranOps:~/kube# kubectl delete pod delete wordpress-deployment-1969810892-z76tv
pod "wordpress-deployment-1969810892-z76tv" deleted
Error from server (NotFound): pods "delete" not found
root@DevImranOps:~/kube# kubectl get pod
NAME                                    READY     STATUS    RESTARTS   AGE
wordpress-deployment-1969810892-1hkkz   2/2       Running   0          7s
root@DevImranOps:~/kube#
```

Figure 85: Alt Text

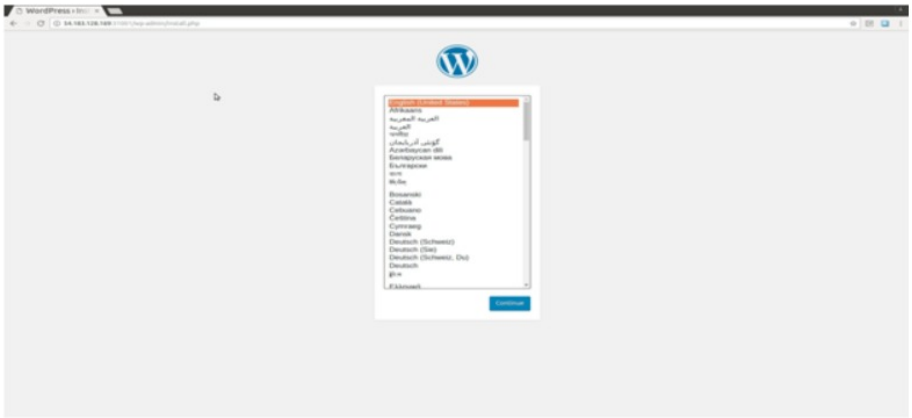


Figure 86: Alt Text